

Trustworthy agentic diagnosis for industrial processes: evidence-graded competing-hypothesis reasoning evaluated against per-fault literature baselines

ZhangHaoZheng^a

*^aInstitute of Advanced Technology, University of Science and Technology of China, Hefei
230088, China*

Abstract

Large language model (LLM) based diagnosis promises root-cause analysis of industrial processes without task-specific model training, yet two obstacles keep it out of engineering practice: conclusions that cannot be audited, and evaluation protocols that score sample-level classification while engineers need scenario-level, mechanism-grounded root causes. We present the Industrial Deep Diagnostic (IDD) pipeline: nine-stage agentic orchestration (ontology, deterministic statistics, competing-hypothesis reasoning, physical verification, quality gates with bounded repair) with evidence grading (levels L1–L7), a four-condition anti-spurious-correlation filter, and a machine-checked evaluation framework with stage-authored, provenance-labelled artifacts. On a 12-scenario benchmark built from three public datasets (SKAB, Tennessee Eastman Process, IndPenSim)—nine documented fault scenarios, six of them on TEP, plus three normal controls—IDD resolves 6/9 faults to a single root cause (Top-1 66.7%, Wilson 95% CI [35.4%, 87.9%]; 5/9 under the pre-revision keyword set) and ranks the true mechanism among the

Email address: zhanghaozheng@ustc.edu.cn (ZhangHaoZheng)

candidates in two more; the ninth (TEP IDV3) is reported as a mechanism-level miss; the three scenarios whose data cannot discriminate mechanisms return capped `COMPETING_SET` verdicts naming the missing measurement; and no control raises a false alarm. A seeded same-era re-execution flipped one fault’s verdict state while agreeing on mechanism; under that draw the resolved count would be 5/9—a sensitivity reported at the headline, not only in the stability analysis. On the TEP subset IDD is Top-1 correct on 5/6 faults without candidate lists (4/6 under the pre-revision keyword set). A model-controlled ablation bounds every accuracy claim: a bare call to the same model family over the same blind digest resolves 9/9 faults—more than the pipeline—so the pipeline’s margins on documented faults are auditability, calibrated uncertainty, and gated process, not accuracy. Internal consistency is machine-verified against the released execution proofs.

Keywords: Fault root-cause analysis, Large language models, Multi-agent systems, Competing-hypothesis reasoning, Evidence grading, Reproducible benchmark

1. Introduction

Root-cause analysis (RCA) of industrial process anomalies is a knowledge-intensive activity: the engineer must decide *which* of many correlated sensor deviations represents the true underlying cause, justify that judgement with evidence, and communicate it with a calibrated degree of confidence. Three long-standing obstacles make automation of this task unsatisfying in practice.

First, single-LLM diagnosis is unstructured and unauditable. Recent evaluations of LLM-based fault explanation on the Tennessee East-

man Process (TEP) [1] show that large models can articulate plausible mechanisms, but their answers arrive as free-form text with no machine-checkable structure, no separation of facts from inference, and no mechanism that prevents a confident-sounding conclusion from resting on a spurious correlation. A recent review of agentic approaches to process monitoring and fault diagnosis concludes that evidence for LLM/multi-agent fault diagnosis remains scarce [2]: the direction is promising but lacks engineering-grade pipelines.

Second, classical data-driven fault detection and diagnosis (FDD) optimises the wrong target. Principal-component methods [3], deep classifiers [4, 5], and their benchmarks [6] score *per-sample* detection or classification [7]. An engineer, however, asks a *per-episode* question: given this anomalous run, what is the root cause, how confident are we, and what evidence supports it? A 99%-accurate classifier that returns a fault label without mechanism, evidence, or calibration does not answer that question. The detection–diagnosis gap is not merely theoretical: classical detectability studies place the documented disturbances IDV3, IDV9 and IDV15 of the fifteen-fault TEP catalogue at the detection limit, with IDV4 weakly detected by the T^2 statistic [8, 9], and the published LLM baseline’s own PCA implementation leaves all four of these faults unscored [1].

Third, correlation is not causation. In recycled process loops almost every pair of variables is correlated through control action or material balance. An RCA method that treats strong correlation as causation will confidently report control-loop artifacts as root causes. Despite decades of causal-inference literature [10, 11], contemporary LLM-diagnosis studies rarely include explicit anti-spurious safeguards.

This paper addresses the three obstacles with the Industrial Deep Diagnostic (IDD) pipeline and a benchmark built to evaluate it honestly. Our contributions are:

1. **A quality-gated agentic diagnosis pipeline** that couples deterministic statistical tooling with LLM reasoning under a competing-hypotheses protocol (Section 4), bounded repair loops, and automatic physical verification, so that every conclusion is decomposable into hypothesis, evidence, exclusion, and confidence.
2. **An evidence-grading and calibration scheme** (levels L1–L7 with gate propagation) that bounds reported confidence by the weakest supporting evidence level. In the corpus none of the determined verdicts is wrong, and where the data cannot discriminate between surviving mechanisms the pipeline returns capped `COMPETING_SET` verdicts with named discriminating measurements instead of forced conclusions (Section 7).
3. **A reproducible evaluation framework**—truth isolation, data fingerprinting, artifact-provenance labels, recorded execution traces, and a machine-checked artifact-consistency gate—in which a scenario is graded only if its complete artifact set was authored by gated pipeline stages and the execution proof passes; an internal integrity audit removed a scripted shortcut that could have fabricated artifacts and agent events, and the benchmark was re-executed end to end under the tightened contract (Section 5).
4. **A 12-scenario benchmark** on three public datasets (SKAB, TEP, IndPenSim) whose TEP subset carries scored per-fault baselines from

FaultExplainer [1] on four faults (the remaining two are scored by IDD precisely where FaultExplainer’s PCA implementation excludes them). Executed on the full benchmark, IDD resolves 6/9 faults to a single root cause and ranks the true mechanism among the candidates in two more; on the TEP subset it is Top-1 correct on 5/6 faults without candidate lists, matching the strongest o1-preview baseline on the shared detectable faults and converting the GPT-4o baseline’s IDV1 miss, while raising zero false alarms on the controls (Sections 7–8); the graded runs executed inline under each stage’s own skill protocol (Section 5).

Section 2 positions the work against data-driven FDD, LLM-based diagnosis, and multi-agent orchestration research. Sections 3 and 4 detail the architecture and methodology. Section 5 defines the evaluation framework. Sections 6–8 present the benchmark and comparisons, and Section 9 discusses limitations and deployment prospects.

2. Related Work

2.1. Data-Driven Fault Detection and the Benchmark Gap

Statistical process monitoring on the Tennessee Eastman Process (TEP) established the dominant evaluation template [12, 7]: principal-component analysis (PCA) and projection methods with T^2/Q statistics [3], later extended by deep anomaly detectors [13, 4] and adversarial robustness studies [5]. These works score *sample-level* detection or *fault-class* accuracy. Critically, they do not evaluate root-cause *mechanisms*: a classifier that labels a run “IDV(4)” provides the class identity but neither a causal mecha-

nism, an evidence trail, nor a confidence statement. The classical detection ceiling is documented: PCA-based monitoring achieves near-zero detection rates on IDV3, IDV9 and IDV15 and only weak T^2 detection of IDV4 [8, 9]. Benchmark methodology has drawn scrutiny: random segment splits inflate reported AUROC—by up to 15.3 percentage points of Macro-AUROC under segmentation-level leakage in Vieira et al. [14], with a lineage formalised by Hendriks et al. [15]—motivating our entity-level scenario splits (Section 5). Scenario-level root-cause localisation itself has a classical lineage that IDD must be read against: Granger-causal and transfer-entropy methods localise propagation directions in plant-wide oscillations [10, 11], signed-directed-graph reasoning performs mechanism-chain elimination [16], and digital-twin approaches mirror model-based diagnosis [17].

2.2. LLM-Based Industrial Diagnosis

FaultExplainer [1] is the closest published system: it couples PCA-based detection with LLM-generated fault explanations on TEP. Its evaluation uses two prompt regimes. In the root-causes-included regime the prompt carries the documented root-cause list, the model outputs a top-3 candidate list, and alias matches are accepted (alias classes: IDV1/2/8, IDV3↔IDV9, IDV4↔IDV11↔IDV14, IDV5↔IDV12↔IDV15); here GPT-4o resolves 7/11 and o1-preview 9/11 of the 11 PCA-detectable faults (the paper’s own PCA implementation excludes IDV3, IDV4, IDV9 and IDV15 from scoring). In its General-Reasoning regime—no candidate list, designed to simulate unseen faults—both models place the correct or a related cause in the top-3 for 8/11 faults under lenient “related” grading (FE’s own text names faults 10 and 13 as failing yet reports 8/11; we transcribe the reported figure).

Three protocol properties therefore matter wherever we compare: (i) FE’s outputs are top-3 lists with alias/related acceptance, whereas IDD commits to a single mechanism verdict scored by strict mechanism keywords; (ii) the two PCA-excluded faults in our subset (IDV3, IDV4) are scored by IDD; (iii) FE’s alias policy grades a cooling-water-family answer as correct for IDV11 and IDV14 alike, whereas IDD separates the three forms. Gong et al. [18] orchestrate multiple LLM agents with difficulty-aware routing, but evaluate on a multiple-choice QA benchmark (FailureSensorIQ [19]) rather than on real time-series data.

Liu et al. [20] combine scene graphs, fine-tuned Llama-3 and augmented reality for machine-tool maintenance, and causal-graph RCA has been studied for time-varying processes [21] and knowledge-map-based group decision settings [22]; a label-free evaluation scheme for diagnosis models has been proposed in the same venue [23]. Activity in this journal and in contemporaneous preprints confirms the momentum and the gaps. In this journal, CausalKGPT couples an industrial causal knowledge graph with an LLM for quality-problem cause analysis [24]; FD-LLM fine-tunes a large language model for equipment fault diagnosis [25]; Wen et al. orchestrate LLMs for human-machine collaborative troubleshooting [26]; and Xu et al. pair a deep digital twin with a vision-language model for multi-scenario fault diagnosis [27]. Closest to our setting, the contemporaneous AgentRCA (July 2026) builds an agentic RCA system that maintains a hypothesis table over tool-grounded evidence, reporting 40.0% Top-1 and 61.5% Top-2 on ten TEP faults—excluding IDV3/9/15 for the classical detectability reasons we discuss [28]; its output is a ranked hypothesis list without calibrated confidence,

Table 1: Positioning Against the Closest Published Systems. Each row scores a published or contemporaneous system on real time-series input, scenario-level root-cause analysis, an evidence trail, calibrated confidence, and a reproducibility gate; IDD is the only system that combines all five.

System	Input	Scenario-level RCA	Evidence trail	Calib.	Repro. gate
FaultExplainer [1]	TEP series	partial (LLM text)	partial	no	no
Gong et al. [18]	MCQA only	no (QA acc.)	no	no	no
PCA/Deep FDD [3, 4]	series	no (labels)	no	n/a	partial
Causal RCA [21, 22]	series	partial (graph)	partial	no	no
IDD	series	yes (3-state)	yes (L1–L7)	yes	yes

and its evaluation, like FE’s, stops short of an external truth-comparison harness. Table 1 summarises the positioning: among published and contemporaneous systems, none combines (i) real time-series inputs, (ii) mechanism-level root causes with evidence trails, (iii) calibrated three-state conclusions, and (iv) a machine-verifiable evaluation contract—elements (iii) and (iv), in particular, are absent from every system cited above, with AgentRCA naming uncertainty communication as future work.

2.3. Multi-Agent Orchestration Frameworks

Generic agent frameworks (AutoGen [29] and commercial derivatives; ReAct-style reasoning–acting loops [30]) provide message-passing and role prompting but leave audit, evidence policy, and evaluation contracts to the

application. Our position, following the practice of the reviewed agentic-FDD literature [2], is that industrial diagnosis requires these as *first-class, machine-checked* pipeline elements rather than prompt conventions; Section 3 describes how IDD enforces them through checkpoints, bounded repair loops, and an event-log execution proof.

2.4. Trust, Automation Levels, and Intellectual Ancestry

Although not framed in these terms, IDD’s design decisions sit on well-mapped ground. The three-state verdict protocol is a level-of-automation and trust-calibration decision—what the system concludes versus what it escalates to humans—in the sense of Parasuraman et al.’s types and levels of automation [31] and Lee & See’s appropriate-reliance account of trust [32]: a capped `COMPETING_SET` is precisely a calibrated-transfer design. The competing-hypotheses discipline descends from Heuer’s Analysis of Competing Hypotheses [33], the four-condition anti-spurious filter from Hill’s causation criteria [34], and the `COMPETING_SET/NEEDS_DATA` abstention option from Chow’s recognition-with-reject tradeoff [35]; the L1–L7 grounding check doubles as a faithfulness proxy in the sense of XAI evaluation. The ontology-asset layer continues a process-engineering ontology lineage (OntoCAPE [36]), repurposed here as a reusable, publishable asset of an agentic pipeline. IDD’s contribution with respect to this ancestry is a domain-specific, machine-checked instantiation of these frameworks inside an agentic pipeline—not their invention.

3. System Architecture

3.1. Execution Architecture

Fig. 1 shows the skill execution architecture as five machine-checked planes. Client surfaces (the web application, CLI drivers, and a CI harness) submit jobs through the engine layer: fourteen interchangeable CLI/RPC engines behind one uniform event contract, with pre-flight probing and typed error codes. The orchestrating main agent holds the domain knowledge as *skill packages* and dispatches each pipeline stage by passing absolute paths (`SKILL_PATH`, `RUN_DIR`); sub-agents communicate only through workspace artifacts, never through conversation context.

Three scoping clarifications make this layer map onto the released software precisely. First, the package contains 18 skill packages executed by 14 specialised agents, but the *diagnostic path* reported in this paper (Steps 2–8.5) dispatches eight specialist sub-agents—context-builder, data-processor, diagnostician, judge, report-reviewer, reporter, html-visualizer, html-reviewer—with the VLM visual analyzer dispatched by the data-processor as a nested agent call inside its Phase 5.5, not as a separate orchestrator dispatch (the orchestrator’s dispatch table labels this sub-step 3+3.3), while the remainder form optional layers outside the benchmarked path (Section 3.2). Second, a skill package is not a prompt alone: every dispatched sub-agent package carries the protocol checklist (`references/agent-protocol.md`), and most also ship the JSON Schemas and validation scripts that machine-check their artifacts before the stage may complete and produce its sub-agent’s deliverable—all eight dispatched packages carry the checklist, while six of the eighteen do not—the or-

chestrator package itself, the Step-0.5 pre-processor, and four support packages (enhancement, RAG knowledge building, dashboard rendering, skill fitness)—the structural difference from prompting a model directly. Third, the engine layer is what makes the pipeline harness-portable: all fourteen engines (claude, omp, codex, opencode, dsh, gemini, copilot, cursor, crush, goose, qwen, pi, hermes, mock) implement one start/stream/close contract, so an engine substitution is a configuration change rather than a re-implementation—and is pre-registered as an ablation axis. Every stage transition appends a structured event to the audit plane, which the log checker and the finalisation gate reduce to a single machine verdict; grading (Section 5) reads only artifacts that survived this plane.

3.2. Knowledge, Web, and Enhancement Layers

Three layers sit beside the diagnostic path and are exercised, degraded, or idle in the benchmark exactly as recorded.

Semantic-knowledge layer. A retrieval microservice (FastAPI + ChromaDB, `rag-retrieval-engine/`) supplies semantic understanding for ontology construction and is the only sanctioned source of level-L6 external evidence; the orchestrator probes it with a three-second health check at Step 0. When it is unavailable, the ontology builder falls back to the shipped `parameter_to_physics.json` mechanism map plus web search, and the degradation is recorded in the run events. In the benchmark corpus the channel’s engagement varied run by run: one run retrieved successfully (the canonical SKAB valve run pulled 21 chunks); the TEP-era runs `tep_d00` and `tep_d01` and the IndPenSim runs `batch001` and `batch093` record explicit probe-failure or empty-retrieval fallback events; the canonical TEP IDV14

run’s `rag_deep_understanding.json` records `rag_available=false` with a fallback reason—no TEP-domain knowledge was retrieved, and the ontology fell back to L5 standard TEP semantics plus L4 data self-description—rather than a successful retrieval; and the remaining canonical runs bypassed the channel silently through the deterministic ontology store-reuse fast path: canonical `tep_d03`, `tep_d04`, `tep_d07`, and `tep_d11` took that silent fast path, with no fallback event in their logs, and the two other SKAB runs (cavitation and the normal control) lack the RAG artifact via the same fast path. No L6-tagged item enters any surviving chain as decision evidence—the single item carrying the combined L5/L6 external marker in the released corpus (the canonical TEP IDV14 run) is marked corroboration-only—so no verdict rests on external evidence.

Ontology asset store. Ontologies are published to a portable asset store (`ontology_store.mjs publish`) keyed by a schema fingerprint; a later scenario whose variable schema matches is resolved by deterministic fast-reuse, which copies the validated ontology verbatim into the run directory in seconds instead of re-deriving it. The store is also the mechanism behind the sibling-scenario reuse reported in Section 9: the reuse path is deterministic and therefore does not affect the benchmark’s truth-isolation contract (it reuses variable semantics, never scenario verdicts).

Integration surface. The web application is an Express backend (port 3210) with a Vue 3/Vite frontend (port 5180), an SSE/WebSocket live stream, and SQLite in WAL mode. Its diagnosis endpoint accepts an engine selection and returns typed pre-flight errors—an unknown engine yields 400 (`HARNESS_UNKNOWN`) and a registered-but-unusable engine 409

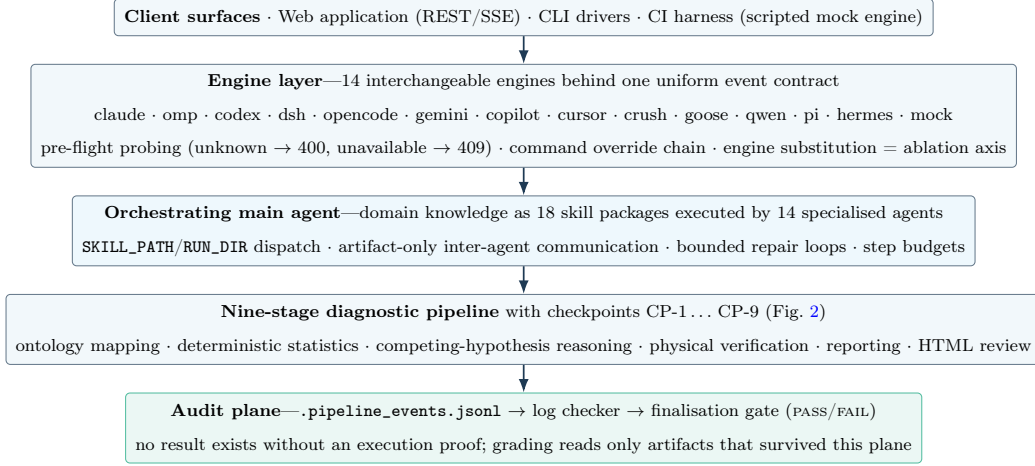


Figure 1: Skill execution architecture. Fourteen interchangeable engines expose one uniform event contract to the orchestrating agent, which dispatches the nine-stage pipeline from 18 skill packages; every transition is recorded on the audit plane whose log checker and finalisation gate produce the machine execution proof consumed by the benchmark grader.

(HARNESS_UNAVAILABLE)—with probing and the real spawn sharing one code path. The optional enhancement layer (E0–E8, deterministic scripts orchestrated by a dedicated agent) post-processes a completed run for reporting depth; it is outside the benchmarked path and not exercised by any scored scenario.

3.3. Pipeline Overview

IDD organises diagnosis as a nine-stage pipeline executed by the specialised agents backed by the skill packages (Fig. 2; the benchmark’s execution form—inline, protocol-faithful stage execution—is disclosed in Section 5). The main orchestrator performs setup and data ingestion (Step 0/1) with adaptive preprocessing; a context-builder agent constructs

a process ontology with variable semantics, physical principles, and confounder annotations (Step 2, checkpoint CP-2/3); a data-processor agent runs the deterministic statistical stage—per-variable anomaly profiling, cross-domain correlation, lag-compensated cross-correlation, batch integrity checks—producing a machine-readable analysis conclusion (Step 3, CP-4); a vision–language-model (VLM) agent reads rendered charts as visual evidence (pipeline Step 3.5 is executed inside the data-processor agent as its internal Phase 5.5; the paper’s four numbering schemes—pipeline Steps 0–9, agent Phases 0–7, reasoning segments R1–R8, method stages 1–6—are independent). The diagnostician agent then performs competing-hypothesis reasoning over the statistical and ontological evidence (Step 4, CP-5). Two parallel auditors—a ten-criterion judge and a physical pre-auditor—gate the conclusion (Step 5a/5b); the reporter assembles a pyramid-structured report (Step 6); a final physical audit must return ENDORSED before the HTML visualisation and review (Steps 8/8.5, CP-9 at the review gate) and the evidence-closure finalisation (Step 9).

Why nine stages. The decomposition follows three rules, each of which the model-controlled ablation independently supports (Section 8.3). Deterministic work is pushed down to scripts: statistics, hashing, schema checks, and the finalisation inventory run as versioned code so they are byte-reproducible, and only semantic judgement (ontology, hypotheses, causal framing, prose) is left to agents—this split is what makes the deterministic layer defensible when a bare model call is already accurate. Data description precedes diagnosis: the ontology and the statistics digest are produced before any hypothesis is generated, which is what forces the diagnose-only-

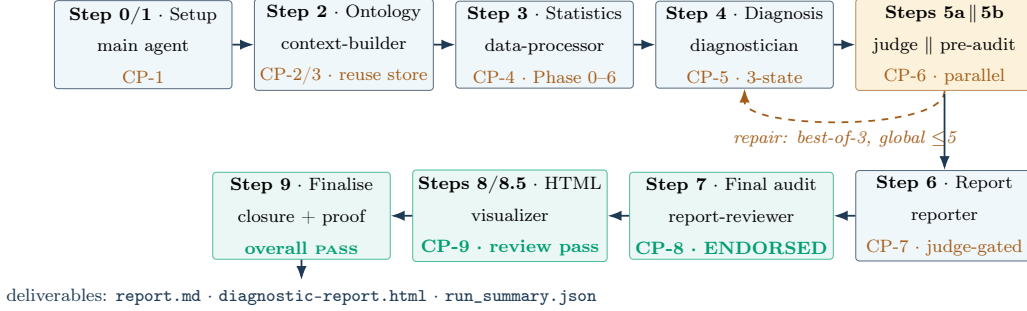


Figure 2: Diagnostic pipeline flow. Solid arrows: strictly sequential stages, each gated by a machine-validated checkpoint (amber tags); the judge and the physical pre-audit (Steps 5a || 5b) are the only mandatory parallel stages; the bottom deliverables row reads right to left, mirroring the top row. The dashed amber edge is the bounded repair loop: arrows re-enter the gated diagnosis stages under the bounded-repair counters (best-of-3 per fault, global cap 5, anti-oscillation confidence cap 0.50), not an unbounded cycle. Steps 7 and 9 are hard gates: the final audit must return ENDORSED and the finalisation gate must return PASS before the deliverables exist. Green boxes mark the hard gates (Steps 7/8.5/9).

what-the-data-supports behaviour observed in the capped verdicts. Audit and delivery are gated last and separately: two independent auditors (judge and physical auditor, the pipeline’s only mandatory parallel step) review the reasoning before a report exists, and the HTML deliverable is built manifest-first and reviewed independently, so the final artifacts cannot diverge from the audited verdict. The stage boundaries themselves are the artifact hand-offs in Table 2, and the mechanical predicates of the checkpoints are listed in Table 3.

3.4. Execution Layer: Interchangeable Engines With a Uniform Contract

Reasoning agents are executed by any of fourteen interchangeable engines (Claude Agent SDK, OMP RPC, Codex app-server, DeepSeek/Hermes/Qwen

ACP dialects, OpenCode, and one-shot CLIs including Gemini CLI, Copilot CLI, Cursor CLI, Crush, Goose, and pi) behind a single contract (`startDiagnosis`, `startSessionChat`, `parseStreamEvent`, `registerChild`, `closeQuery`), realized as a twelve-entry capability table (`engines.mjs`) plus the two first-party wiring entries in the service dispatch table. Engine availability is probed and validated before execution with a three-state contract (unknown engine 400, unusable engine 409 `HARNESS_UNAVAILABLE` with human-readable diagnostics); a scripted in-process mock engine allows the full pipeline to be exercised in CI without external dependencies. Because every engine emits the same standardised event stream, the benchmark harness consumes any compliant engine’s event stream identically; diagnostic quality across engines is an empirical ablation question, and engine substitution remains a designed but not-yet-exercised ablation axis (Section 9).

3.5. Audit Infrastructure

Every stage transition appends a structured event to `.pipeline_events.jsonl`; a log checker validates the start and completion records of every agent against the required contract, and a finalisation gate aggregates inventory, schema, closure, judge-cross-audit, and event-archive checks into a single PASS/FAIL verdict. A run that fails any critical check cannot be graded, which is the property the benchmark relies on: *no result exists without an execution proof*. The gate is exercised, not decorative: during the benchmark execution it rejected complete-looking runs on contract violations (deliverable drift between the run summary and the diagnosis, a missing visual-analysis provenance signature, a non-canonical report outline), and the affected stages were re-executed until the gate passed. Events are pipeline-internal records,

not externally signed attestations; the threat model and its limits are discussed in Section [9](#).

Table 2: Module contracts of the diagnostic path: every stage, the agent that authors its artifacts, the artifacts it consumes and produces (filenames as released; all paths are relative to the run directory), and its checkpoint. The table lists each stage’s contracted deliverables; runs additionally hold auxiliary stage-authored files. Checkpoint predicates: Table 3.

Stage	Agent	Consumes	Produces	Gate
Setup/inspect (0/1)	main agent	scenario CSV, user context	00_input/input_manifest.json, user_context.json, run_config.json	CP-1
Ontology (2)	context-builder	input_manifest.json, RAG digest, process description	01_ontology/ontology.json (+00_input/clarification_needed.json)	CP-2/3
Statistics (3)	data-processor	ontology.json, cleaned data	02_processed/data_analysis_conclusion.json (mandatory handoff), validate_report.json, 03_figures/	CP-4
Visual evidence (3.5)	evi- vlm-visual-analyzer (inside data-processor Phase 5.5)	03_figures/, image_captions.json	03_figures/visual_analysis.json	—
Diagnosis (4)	diagnostician	analysis conclusion, ontology, digests	04_diagnostics/{diagnosis, evidence, confidence, reasoning_chain}.json	CP-5
Judge pre-audit (5a/5b)	judge report-reviewer	04_diagnostics/*	05_review/judge_feedback.json 05_review/optimizer_preflight.md	CP-6
Report (6)	reporter	all upstream artifacts	report.md, run_summary.json	CP-7
Final audit (7)	report-reviewer	report.md, artifacts	optimizer.md containing ENDORSED	CP-8
HTML build (8)	html-visualizer	artifacts, design system	render_manifest.json → diagnostic-report.html → html_selfcheck.json	—
HTML view (8.5)	re-reviewer	page + manifest	05_review/html_review.json (verdict=pass)	CP-9
Finalise (9)	deterministic script	all of the above	pipeline_finalize_report.json (PASS), evidence_closure_report.json	—

Table 3: Checkpoint gates CP-1...CP-9. Every predicate is mechanical—file existence, JSON-schema validity, or a numeric lower bound—so the gates can be re-run by a reader without domain judgement; a failing predicate blocks the transition and routes to the stated repair action.

CP	Transition	Machine predicate	Failure action
CP-1	1→2	<code>input_manifest.json</code> + <code>user_context.json</code> + <code>run_config.json</code> exist	back to Step 0
CP-2	2→2.5	<code>ontology.json</code> ≥ 1 KB and schema-valid	re-run ontology builder
CP-3	2.5→3	<code>clarification_status</code> $\in$ { <code>AUTO_RESOLVED</code> , <code>USER_CONFIRMED</code> }	resolve clarification
CP-4	3→4	<code>data_analysis_conclusion.json</code> present and generated plots > 0	re-run data-processor
CP-5	4→5	four diagnosis outputs schema-valid and quality-checked	repair diagnosis
CP-6	5→6	judge repair summary written; pre-audit reports no FATAL finding	repair (best-of-3)
CP-7	6→7	<code>report.md</code> + <code>run_summary.json</code> present	re-run reporter
CP-8	7→8	<code>optimizer.md</code> contains <code>ENDORSED</code>	repair loop
CP-9	8.5	<code>diagnostic-report.html</code> ≥ 5120 B and review verdict pass	re-run html-visualizer

4. Diagnostic Methodology

4.1. Problem Formulation

A scenario supplies a recorded window D_S of p process variables and, optionally, user context; the task is to produce a verdict $v \in \{\text{DET}, \text{CS}, \text{ND}\}$ with a mechanism-level root cause, a confidence $c \in [0, 1]$, and an evidence chain. The diagnostician forms a hypothesis set $H = \{h_1, \dots, h_k\}$, $k \geq 3$, where each h_i carries a mechanism class m_i , a set E_i of evidence items, and a contradiction set X_i . Two definitions govern the protocol. The *evidence level* of a hypothesis is the weakest level in its chain, $\ell(h_i) = \min_{e \in E_i} \text{level}(e)$ with $\text{level}(e) \in \{L1, \dots, L7\}$ (Section 4.3)—a conclusion cannot be stronger than its weakest citation. The *confidence ceiling* is the minimum of the five-factor assessment and every protocol cap applicable to the verdict state,

$$c \leq \bar{c}(v) = \min(F, \mathcal{C}(v)), \quad (1)$$

where F is the five-factor assessment and the cap function $\mathcal{C}(v)$ evaluates to the smallest applicable protocol cap: 0.70 for CS, tightened to 0.65 for its INDISTinguishable subclass, and 0.50 whenever the anti-oscillation rule (code name `PARAM_AMBIGUITY`) is active—all three are documented in the agent-side protocol. The agent-side protocol documents no DET cap, but the machine grading gate enforces confidence ≤ 0.90 for every non-CS state and ≤ 0.70 for CS (calibration check R4 of the released rubric), so the effective cap set $\mathcal{C}$ includes 0.90 for DET at the gate (the gate’s flat non-CS 0.90 ceiling also covers ND); reason-coded per-run ceilings may still bind (the released schema enumerates the reasons: indistinguishable competing set, unsorted-data lag claims, product-specific-only evidence, all-criteria-verified,

direct measurement); e.g. the canonical IDV4 run binds its DETERMINED verdict at 0.75 with reason `DIRECT_MEASUREMENT`. Elimination is quantitative: h_i is eliminated when its contradictions outweigh its support, $|X_i| > |E_i|$ under the evaluator’s weighting—the protocol rule the diagnostician agent applies when eliminating, with each elimination additionally recording a revival condition; the machine-checked parts of the contract are the minimum of two eliminations overall, the schema floor of exclusion confidence ≥ 0.90 , and the per-elimination citation of the contradicting evidence. The three-state verdict is then the decision function

$$v = \begin{cases} \text{DET}, & |H_{\text{surv}}| = 1 \text{ and every elimination is evidenced,} \\ \text{CS}, & |H_{\text{surv}}| \geq 2 \text{ and no channel separates the survivors,} \\ \text{ND}, & \ell(h^*) < L5 \text{ or a required measurement is absent,} \end{cases} \quad (2)$$

with the reported confidence satisfying $c \leq \bar{c}(v)$ (Eq. (1)). The observed 0.58–0.65 band reflects this INDISTINGUISHABLE subclass. The five-factor assessment is the protocol’s confidence decomposition over statistical strength, physical plausibility, temporal evidence, absence of confounds, and symptom completeness (weighted 25/25/20/20/10, totalling 100, in the released `confidence.json`); the three states are protocol terms throughout this paper and carry no everyday meaning: DET asserts a single evidenced mechanism, CS asserts that the present data cannot separate the survivors and names the measurement that would, and ND declares the evidence insufficient. Algorithm 1 states the whole procedure as executed, with deterministic (script) and semantic (agent) steps marked; the ablation of Section 8.3 is the empirical test of this split.

Algorithm 1 IDD diagnosis of scenario S (script = deterministic code, agent = semantic reasoning; gates as in Table 3)

- 1: **script:** fingerprint D_S , compute statistics digest, write blind brief ▷
Step 0/1
 - 2: **agent:** $O \leftarrow$ ontology (brief) or **script**-side fast-reuse from the asset store ▷ Step 2, CP-2/3
 - 3: **agent:** $A \leftarrow$ data-processor (D_S, O); plots, validation report ▷ Step 3, CP-4
 - 4: **agent:** form H , attach (E_i, X_i) with evidence levels ▷ Step 4
 - 5: **agent:** eliminate while $|X_i| > |E_i|$; compute v by Eq. (2)
 - 6: $c \leftarrow \min(\text{five-factor assessment}, \bar{c}(v))$ ▷ Eq. (1)
 - 7: **if** judge score < 90 **then snapshot** best round; repair in declared scope (≤ 3 rounds, global cap 5)
 - 8: **end if** ▷ CP-6
 - 9: **agent:** report; **verify:** physical audit must return ENDORSED ▷ Step 6/7, CP-7/8
 - 10: **agent:** HTML manifest-first, then independent review ▷ Step 8/8.5, CP-9
 - 11: **script:** finalise—the grader’s 14-artifact completeness set, schemas, closure, event log $\Rightarrow$ PASS; else the run cannot be graded (the Step-9 finaliser separately executes its own ≈ 45 -check inventory)
-

4.2. Competing-Hypotheses Protocol

The diagnostician agent must generate at least three candidate hypotheses per fault scenario, assign each a mechanism class (e.g., fluid restriction, cavitation, rotor dynamics, feed-composition step, cooling dis-

turbance, sensor drift), collect for each an evidence list with explicit levels, identify contradictions, and eliminate hypotheses whose contradictions are stronger than their support. The protocol admits exactly three conclusion states: DETERMINED (a single surviving hypothesis, bounded by the five-factor assessment; reason-coded per-run ceilings may further bind it), COMPETING_SET (≥ 2 survivors that the data cannot discriminate, with confidence capped at 0.70; observed caps in this corpus are 0.58–0.65 and the discriminating measurement is named), and NEEDS_DATA (evidence insufficient; missing data itemised). A fourth, implicit state—normal operation with no confirmed fault—applies to control scenarios. The elimination contract is quantitative: every elimination must cite the contradicting evidence, and every surviving chain must cite evidence numbers present in the statistical digests.

4.3. Evidence Grading and Gate Propagation

Every piece of evidence carries a level L1–L7 (L1 direct measurement, L2 operating documents, L3 statistical analysis, L4 visual evidence, L5 domain ontology/mechanism, L6 external reference, L7 unsupported assumption). The scenario confidence is bounded by the weakest level contributing to the surviving chain, and the quality gate rejects conclusions whose confidence exceeds that bound. This converts “hallucination control” from a prompt instruction into a checkable arithmetic constraint.

4.4. Anti-Spurious-Correlation Filter

Four conditions must hold before a correlation may be cited as causal support: (i) temporal precedence of cause over effect, verified by lag-

compensated cross-correlation; (ii) statistical significance surviving detrending, Simpson-stratification checks, and multiple-testing correction; (iii) a physical mechanism annotated in the ontology connecting the variables; and (iv) absence of contradictions under leave-one-out outlier sensitivity analysis, with production-state (steady/batch-phase) filtering applied before correlation. Correlations failing the filter are recorded but explicitly barred from hypothesis support; definitional pairs (e.g., valve position and the level it controls, $r=1.0$) are classified as control-loop artifacts. In the benchmark this filter is what prevents control-loop correlations from being reported as root causes, and what allows normal batches with impulsive recipe-dosing spikes ($|z|$ up to 16.2) to be graded as healthy rather than faulty.

4.5. Automatic Physical Verification

After the conclusion is drafted, a physics checker recomputes quantitative feasibility (mass/energy balances, thermal and vibration magnitude envelopes) from the cleaned data, ontology, and anomaly report, producing a machine-readable `physical_checks` verdict. When no automated check applies to a scenario’s column semantics, the pipeline falls back to an explicit L1–L5 manual-verification memo; silent omission is not permitted.

4.6. Quality Gates, Repair Loops, and Calibration

A ten-criterion judge scores the diagnosis (threshold 90); a parallel physical pre-audit reviews mechanism chains; failures trigger bounded repair (per-fault rerun ≤ 3 , global cap 5, with anti-oscillation confidence capping at 0.50). The final physical audit must return ENDORSED. Ceiling compliance is then auditable end-to-end: every scenario grading records whether the reported

confidence respects the protocol ceiling, and *overconfidence* (DETERMINED and wrong) is a first-class error metric.

Repair is engineered rather than incidental, and the released governance is what keeps a stochastic reasoner bounded. Each repair round snapshots the best-scoring attempt (`best_round` copy of the diagnosis set); the second and third rounds run in a declared *repair scope* that recomputes only the implicated artifacts and marks the rest as carried over, so a fix cannot silently rewrite evidence that was not found defective. The loop never halts the run on score alone: if the threshold is still unmet after the budget, the pipeline delivers a best-effort report with a gate-enforced capped verdict (and the cap recorded), because failing to deliver is worse than delivering with a disclosed confidence. Two further disciplines are enforced as machine-checkable rules rather than prompt advice. The *data-truth mandate* requires every number in a diagnosis to be reproducible from the raw data (chain items carry derived markers and a NOT-APPLICABLE fallback), and the ten-item red-light list makes the failure modes auditable: no forced single verdict from a competing set, no global correlation cited as causal evidence, no conclusion without an evidence-level tag, no HTML outside the manifest. Violations are what the judge is instructed to flag, which is how the audit plane of Section 3 remains a check rather than a claim.

5. A Reproducible Evaluation Framework

IDD’s benchmark framework (released as the project’s `experience/` evidence repository and `docs/benchmark/` protocol) is designed around five contracts, each machine-enforced:

(C1) Truth isolation. Scenario definitions carry ground truth and grading keywords, but the pipeline’s visible inputs contain only the process description and the data. Ground truth never enters the inference context; the grader owns it exclusively. A leakage sentinel machine-checks the contract from both directions: no ground-truth string may appear in any agent-visible brief, and no fault-vocabulary may appear in the run directories’ user-context files. Scenario definitions—including the grading keywords and the allowed verdict set of every scenario—are versioned in the released repository with their edit history, and the calibrated keyword set—over-generic tokens removed (e.g. single-purpose Chinese tokens for temperature and fluctuation), mechanism synonyms added—was applied once, uniformly, to the working scenario definitions before any canonical grading that counted, and grading-side only: keywords never touch execution inputs. Table A3 (Appendix A) records the journal chronology—calibration landing before any canonical grading that counted, one superseded pre-switch batch disclosed rather than hidden, and the journal’s `calibrated` field recording verdict-type compliance, not keyword provenance. A keyword-sensitivity re-scoring of the frozen corpus under the pre-revision keyword sets (driver `keyword_sensitivity.mjs` and addendum `keyword_sensitivity.json`, released under `scripts/benchmark/` and `results/benchmark/`; the calibrated set is pinned at commit 221d061) shifts exactly one scenario in each direction—the added IDV1 synonym gains one hit and a deleted generic token’s spurious IDV3 hit disappears—so Top-1 is 5/9 pre-revision versus 6/9 calibrated (TEP subset: 4/6 $\rightarrow$ 5/6) while top- k stays 8/9. The frozen corpus was scored under the calibrated set, the pre-revision sets remain in the released history,

and we disclose that these keys are author-designed (Section 9).

(C2) Data fingerprinting. Every input file is hashed (SHA-256) into a dataset manifest; the reproducibility gate re-verifies all 153 input fingerprints and the leakage sentinel returns a PASS verdict before any metric is accepted.

(C3) Stage-authored artifacts with provenance. Each pipeline stage writes its own machine-checkable artifacts: the ontology, the statistical analysis conclusion, the four-piece diagnosis, the judge’s ten-criterion feedback, the auditors’ ENDORSED verdicts, the report, the page model (`render_manifest.json`) behind the HTML deliverable, and the reviewer’s independent verdict. Every scored verdict carries an explicit provenance label (`judge-agent-10-criteria`, `auditor-agent`, `html-reviewer-agent`), so downstream consumers can distinguish independently authored judgments from rendered ones.

(C4) Gate-graded scoring. The grader accepts a scenario only if its run directory contains the complete artifact set of an executed pipeline (the fourteen items in Table 4), the event log passes the log checker, and the finalisation gate returns PASS. Otherwise the scenario is reported as *not executed*—a pipeline failure, not a benchmark result. The gate is exercised in practice: an earlier protocol revision allowed a scripted shortcut that expanded a single reasoning note into a full artifact set (complete with synthesised agent events); the internal integrity audit flagged the fabrication, the shortcut was deleted from the codebase, and the entire benchmark was re-executed through the real pipeline. The grader’s completeness gate makes the shortcut permanently unplayable.

(C5) Artifact-consistency gate. A verifier recomputes all aggregate

metrics from the per-scenario gradings, re-checks dataset hashes, verifies case coverage, and requires an execution proof per run; any drift fails the evaluation. The gate verifies the internal consistency of the recorded execution—recomputed aggregates, hashes, coverage, proofs—and is deliberately not an independent re-execution of the reasoning layer; the released report carries the resulting internally-consistent verdict.

Execution form. The canonical benchmark runs were driven by the project’s released benchmark driver (Section 5), whose harness did not expose a sub-agent spawn tool: each stage executed *inline*, strictly under its own skill protocol (persona, protocol checklist, schemas, validation scripts), and the event logs record this per run—eleven of the twelve canonical run logs carry an explicit marker (a dedicated field on eight runs, disclosure or remediation notes on three; only `skab_cavitation_13` lacks one), and the driver itself exposes no spawn tool, which fixes the execution form for all twelve (the SKAB-valve run records `inline_execution:true` with the harness’s no-spawn-tool reason). A corollary is that consecutive stages may share one model session where the dispatch form would isolate them to artifact-mediated communication; the artifact contracts and gates are unchanged. In this form the one mandatory parallel step (judge and pre-auditor, Steps 5a/5b) also ran serialized. The per-stage contracts, artifacts, checkpoints, and provenance labels are exactly those of Section 3; the dispatch mechanism is the only difference, the sub-agent dispatch itself is exercised by the production web harness rather than by the benchmark driver, and we say so wherever execution form matters.

Table 4 lists the graded artifact set the completeness gate enforces; the

Table 4: The graded artifact set enforced by the completeness gate (C4). A run missing any item is *not executed* by the grader—a pipeline failure, not a benchmark result.

#	Run path	Role
1	01_ontology/ontology.json	process ontology (variables, semantics, mechanisms)
2	02_processed/data_analysis_conclusion.json	statistical handoff to the diagnostician
3	04_diagnostics/diagnosis.json	verdict, hypotheses, eliminations
4	04_diagnostics/evidence.json	evidence inventory with numeric evidence ranks (1–7, the L1–L7 levels)
5	04_diagnostics/confidence.json	per-hypothesis confidence and ceiling audit
6	04_diagnostics/reasoning_chain.json	R1–R8 reasoning chain with falsification conditions
7	05_review/judge_feedback.json	judge sub-agent ten-criterion verdict
8	05_review/optimizer_preflight.md	pre-report physical audit
9	05_review/html_review.json	independent HTML review (verdict=pass)
10	report.md	pyramid report
11	run_summary.json	machine-readable run summary (consistency-checked)
12	render_manifest.json	page model, written before the HTML
13	html_selfcheck.json	HTML self-check against the manifest
14	diagnostic-report.html	HTML deliverable (size re-check rejects files below 5 120 B)

same fourteen filenames appear in the released run directories, which is what makes “the complete artifact set” checkable rather than rhetorical.

Reproducibility semantics. The framework distinguishes two layers explicitly. The deterministic statistical stage is byte-reproducible: identical inputs and code regenerate identical digests, and the released dataset fingerprints pin the inputs. The reasoning layer is reproducible at the *procedure* level: re-executing the protocol re-launches the same gated pipeline on the same blind inputs and re-derives conclusions under the same gates and truth comparison; wording may vary between runs, but every verdict remains bound to the same evidence discipline. We state both layers wherever

a reproduction claim is made, and we do not claim bit-level reproducibility for LLM reasoning.

Scenario splits follow the entity-level discipline recommended by recent benchmark studies [14, 15]: each experimental batch/file appears in exactly one scenario, controls are paired with faults per dataset, and no segment-level splitting is used anywhere.

6. Experimental Setup

6.1. Datasets and Scenarios

We assemble 12 scenarios from three public benchmarks (Table 5): **SKAB** [37], a closed water-circulation testbed with vibration, current, pressure, temperature, and flow channels at 1 Hz—two fault scenarios (inlet-valve throttling—the official label file names suction-line valve restriction; and two-phase-flow cavitation drawn from the “other” group, whose official label names cavitation but is withheld from the scenario description) plus the anomaly-free control; **TEP** [12, 6], with six documented fault scenarios—IDV1 feed-composition step, IDV3 D-feed-temperature step (classically hard to detect), IDV4 reactor-cooling-water step, IDV7 C-header pressure loss, IDV11 random cooling-water variation, IDV14 cooling-valve stiction—and the fault-free baseline as control; and **IndPenSim** [38, 39], a 100,000 L penicillin fermentation simulator, with one documented process-deviation batch (93) and one normal batch as control. Ground truth derives from verifiable sources only: SKAB’s official label file, the established TEP cause table [9], and IndPenSim’s faulted-batch markers. Scenario selection followed a documentation rule rather than random sampling: every scenario

corresponds to a publicly documented fault episode, chosen so that each dataset contributes mechanisms from distinct families plus a paired control; the consequences of non-random selection are discussed in Section 9.

Three design decisions deserve emphasis. First, the TEP subset is chosen so that *every fault maps onto a published per-fault outcome* from FaultExplainer [1], enabling scenario-granular comparison rather than aggregate-only comparison: four faults (IDV1/7/11/14) carry scored baselines, and two (IDV3/IDV4) are scored by IDD precisely where FaultExplainer’s PCA implementation excludes them. Second, IDV3 sits at the classical PCA detection limit and IDV4 is weakly detected [8, 9]—both lie where classical detection and the published baseline stop—so they probe root-cause reasoning at and beyond that limit. Third, IDV4, IDV11 and IDV14 perturb the same physical channel (reactor cooling water) with three different dynamic forms (step, random variation, valve stiction); separating them requires reading the statistics of each recording, not recalling a fault label, which partially mitigates prompt-knowledge contamination.

6.2. Metrics

For fault scenarios we report **resolved Top-1** (a DETERMINED verdict naming the true mechanism; coincides with the FDD community’s correct-diagnosis-rate, CDR), **top-k** (the true mechanism is named within the primary finding or the surviving-hypothesis set that the grader reads; because the protocol mandates at least three generated hypotheses, this approximates candidate-list membership, and for a single-survivor DETERMINED verdict it coincides with resolved Top-1; reported alongside, not instead of, resolved Top-1), **ceiling compliance** (reported confidence within the protocol ceil-

Table 5: Benchmark Composition. Twelve scenarios from three public datasets: each dataset contributes its documented fault scenarios plus one anomaly-free control, with ground truth drawn from official label files, the established TEP cause table, and IndPenSim’s faulted-batch markers.

Dataset	Fault	Control	Truth source
SKAB	2	1	official <code>ground_truth.json</code> (label withheld for the “other”-group case)
TEP	6	1	Downs–Vogel; Chiang et al. [12, 9]
IndPenSim	1	1	faulted-batch markers (batch 93)
Total	9	3	—

ing of Section 4), **verdict-type compliance** (conclusion type within the scenario’s allowed set; a DETERMINED-and-wrong verdict counts as *over-confidence*), and a **protocol-compliance rubric score**—a deterministic, machine-computed checklist (100 points) computed from the pipeline artifacts themselves and measuring conformance to this paper’s own protocol rather than correctness (see Section 9). For control scenarios we report **pass** (no confirmed fault) and **false alarms**. Proportions carry Wilson 95% confidence intervals at $z=1.96$ (the released scorer’s constant). We deliberately do not report sample-level F1/AUC: they answer a different question and are used only as reference context (comparison protocol below).

The rubric’s seven checks are computed strictly from the stage-authored run artifacts: **R1** artifact integrity (ontology, analysis conclusion, and the schema-valid diagnosis four-piece; 15 points), **R2** hypothesis structure (≥ 3 hypotheses, ≥ 2 eliminations each citing substantive contradictory evidence; 15), **R3** evidence grounding (every surviving chain cites at least one number

present in the scenario’s own evidence brief; 20), **R4** calibration (confidence within the protocol ceiling; conclusion type within the scenario’s allowed set; 15), **R5** falsifiability (a surviving hypothesis names the observation that would revise it; 10), **R6** physics verification (automated check valid or explicit manual memo; 10), **R7** execution proof (event log + finalisation PASS + HTML review gate; 15). The pipeline’s internal ten-criterion judge remains the in-pipeline gate (Section 4)—its scores are recorded with provenance and reported as execution-quality evidence—but it is *not* a benchmark metric: benchmark quality scoring is fully mechanical. Two scope statements keep this rubric honest. First, it is a conformance instrument with limited discriminative power: ten of the twelve scenarios score at ceiling because the structural checks cannot fail for any run that has passed the gates—correctness claims rest on the truth comparison, never on the rubric. Second, “strict mechanism-keyword hit” is not mechanism equivalence: the keyword sets are disclosed per scenario, one of them (the IndPenSim process-deviation scenario) includes the generic token “deviation”, and a hit certifies mechanism-family agreement, not identity.

6.3. Comparison Protocol

We distinguish two comparison registers. **Register A (same-task direct comparison)**: methods that output one root-cause conclusion per scenario on the same dataset family; the published FaultExplainer per-fault numbers fall here, at scenario granularity. **Register B (reference only)**: sample-level detection/classification results; they are reported for context and must not be ranked against our scenario-level metrics. Two protocol differences are stated wherever Register A is invoked: FaultExplainer supplies

candidate root causes in its prompt and accepts alias matches, whereas IDD uses no candidates and scores mechanism keywords strictly; and FaultExplainer scores only the 11 PCA-detectable faults, whereas IDD additionally scores PCA-undetectable IDV3/IDV4. Because the scenario sets differ, no significance test is claimed across systems.

6.4. Implementation

The deterministic stages run as versioned scripts (streaming CSV inspection, anomaly profiling, correlation with lag-compensated cross-correlation functions (CCF), validation with Simpson/detrending/outlier-sensitivity/multiple-testing checks, physics checking via the shared scientific Python environment). The reasoning stages execute the full nine-stage agent pipeline under their skill protocols (Section 3); the benchmark driver runs them directly rather than through the engine registry (execution form in Section 5), and every stage transition is recorded in the event log. A complete 12-scenario sweep takes seconds per scenario for the deterministic stage and 56–220 minutes of agent execution per scenario for the full gated pipeline (median ≈ 110 minutes across the twelve canonical runs; event-log spans). The entire chain (prepare, briefs, pipeline-execution verification, gate-graded scoring, rubric, aggregation, artifact-consistency gate, report) is driven by a single entry point with fail-fast stage gates.

Benchmark configuration disclosure. All twelve scenarios reported here were executed on one harness: the ZCode agent CLI (the CLI-driver class of the engine layer, driven directly by the released `scripts/benchmark/zcode_direct_pipeline.mjs` benchmark driver) with

the GLM model family deployed under the provider’s coding-plan configuration (snapshot September 2026). The harness does not expose temperature or top-p control, so provider defaults applied. The vision stage had no vision-capable model in this deployment and executed its documented non-visual degradation in every run (skeleton visual analysis with caption-based L4 fallback). The execution window was 14–15 September 2026, with one graded execution per scenario; per-scenario repair-loop counts are part of the released event logs (eleven scenarios required zero repairs; one control required a single repair round). Base-model capability is a confound for any cross-system comparison: the protocol-versus-baseline contrast of Section 8 holds the prompt regime constant but not the model, and we flag it as such. The pre-registered ablation matrix targeting this attribution question has been partially executed (Section 8.3): a single LLM prompted with the identical blinded statistical digest, a classical PCA baseline on all twelve scenarios, and a FaultExplainer-protocol replication on the same model family. The engine-substitution arm and the filter-disabled arm remain future work. The 18 skill packages (the prompts executed by each agent) are released with the repository. [Appendix A](#) indexes the released per-scenario artifacts.

7. Results

7.1. Overall Performance

Table 6 reports the headline numbers. Executing the full pipeline under truth isolation (one recorded execution per scenario), IDD resolves **6/9** fault scenarios to a single root cause (Top-1 = 66.7%, Wilson 95% CI [35.4%, 87.9%]; 5/9 pre-revision, Section 5) and places the true mechanism

among the candidates in an additional two scenarios (top-k = 8/9); the remaining scenario (IDV3, historically the hardest-to-detect TEP fault [8]) ends in an honest capped `COMPETING_SET`. Ceiling compliance is 9/9, verdict-type compliance 8/9 (the single miss is SKAB inlet-valve throttling, analysed in Section 7.3), and *no* verdict is simultaneously `DETERMINED` and wrong (determined-verdict precision 6/6). All 3 control scenarios pass with **zero false alarms**. All 12 pipeline executions pass the finalisation gate with valid event logs. The deterministic rubric averages **97.9/100**—ten scenarios at 100; the deductions are: SKAB inlet-valve throttling −15 (R4 verdict-type compliance, analysed in Section 7.3) and TEP IDV11 −10 (R6 physics-check coverage)—the in-pipeline judge gate averages 93.1 (range 90–97; all scores are post-threshold by the repair gate’s construction), and the artifact-consistency gate verifies that the evaluation is internally consistent. Fig. 3 summarises the outcomes; Section 8.3 places them against model-controlled baselines.

7.2. Per-Mechanism Breakdown and Within-Family Discrimination

Table 7 lists the nine fault scenarios with the verdict each pipeline execution actually produced, the confidence, the rubric score, and the decisive statistical signature the diagnosis rested on. Three observations deserve emphasis.

Within-family form discrimination. IDV4 (step), IDV11 (random variation) and IDV14 (valve stiction) all perturb the same physical channel—reactor cooling water—yet produce three distinct statistical forms that the pipeline separated correctly and for three different reasons: a sustained controller-offset plateau with a step-dominant transient (IDV4), pure vari-

Table 6: Overall Results on the 12-Scenario Benchmark (full-pipeline execution; one recorded execution per scenario; 5/9 pre-revision; Section 5). The drawn same-era re-execution of TEP IDV11 flipped its verdict state (determined 0.80 $\rightarrow$ capped COMPETING_SET 0.62, mechanism and localisation unchanged); had that outcome been canonical, resolved Top-1 would read 5/9 (TEP subset 4/6) with top- k unchanged (Section 8.4). Rubric deductions: SKAB inlet-valve throttling -15 (R4 verdict-type compliance); TEP IDV11 -10 (R6 physics-check coverage). Judge scores are post-threshold (≥ 90 by the repair gate), averaged over $n=12$ graded scenarios (nine faults + three controls), and carry no benchmark weight. The determined-verdict precision row shares its events with the Top-1 row (non-independent) and conditions on the pipeline’s own verdict choice (capped verdicts are gate-forced, so the determined set is self-selected); its interval is therefore omitted. Wilson 95% intervals at $z=1.96$, matching the released scorer. The top- k row counts scenarios whose surviving-hypothesis set or primary finding names the true mechanism (equivalent to Top-1 for single-survivor determined verdicts).

Metric	Value	95% CI
Resolved Top-1 (DETERMINED and correct; $n=9$)	6/9 = 66.7%	[35.4, 87.9]
True cause within top candidates ($n=9$)	8/9 = 88.9%	[56.5, 98.0]
Determined-verdict precision (correct when DETERMINED)	6/6	—
Ceiling compliance (confidence $\leq$ protocol cap)	9/9 (gate-enforced by construction)	—
Verdict-type compliance (type in allowed set)	8/9	[56.5, 98.0]
Honest capped COMPETING_SET verdicts	3/9	—
Control pass (no false alarm; $n=3$)	3/3	[43.8, 100.0]
Protocol-compliance rubric (author-designed checklist)	97.9/100 mean	ten at 100, one at 90, one at 85
Judge gate mean (in-pipeline, post-threshold ≥ 90)	93.1	range 90–97
Pipeline finalisation gate	12/12 PASS	—
Artifact-consistency gate	verified	—

ance inflation without mean shift (IDV11, variance ratio = 16.45 with $\Delta\text{mean} = 0.006\sigma$), and a stuck-slip limit cycle comprising 80 $>5\sigma$ events at a stable ≈ 30 min period across three synchronised channels, with $\approx$ zero net drift and an actuator leg in phase with the controlled variable ($r=+0.999$ —the wrong sign for a negative-gain loop) (IDV14). Recalling the textbook label “cooling-water fault” cannot produce these three different mechanisms;

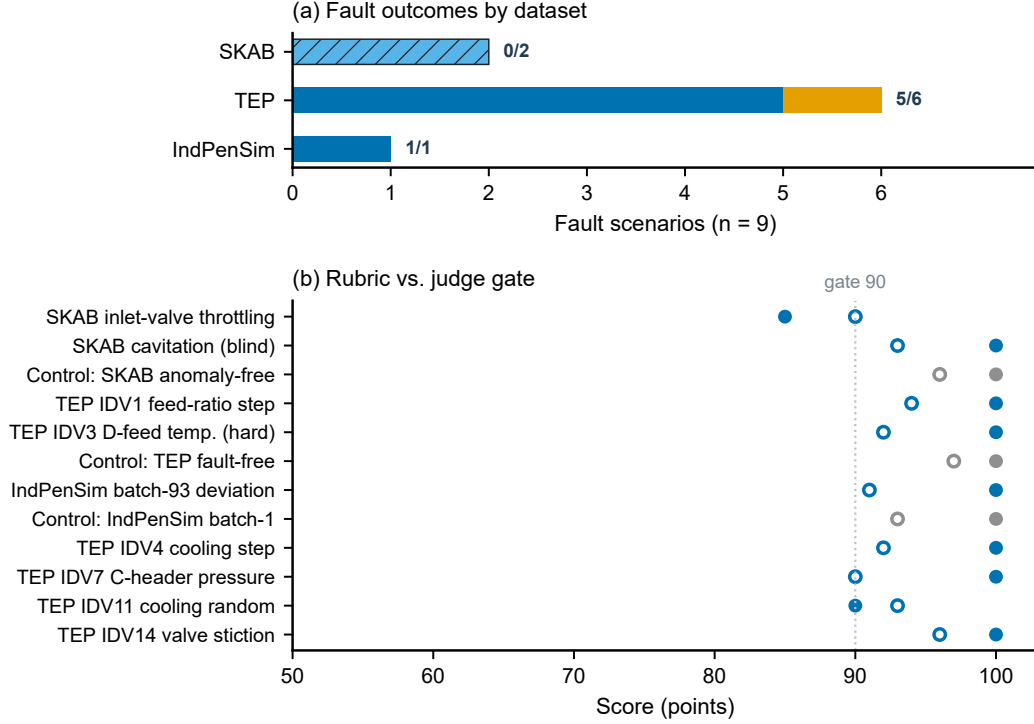


Figure 3: Benchmark outcomes under full-pipeline execution. (a) Fault outcomes by dataset: Top-1-correct DETERMINED verdicts (dark blue), capped COMPETING_SET verdicts whose survivors rank the true mechanism (hatched sky blue), and the single mechanism-level miss (IDV3, amber) whose survivors exclude it; the printed fraction is Top-1-correct over fault scenarios in that dataset (scored under the calibrated keyword set; Section 5), and all three controls pass with zero false alarms (excluded from the bars). (b) Per-scenario deterministic rubric (filled markers) against the in-pipeline judge gate (open markers); dark blue = fault scenarios, grey = controls; the dotted line is the judge’s 90-point gate. All 12 executions pass the finalisation gate, and the artifact-consistency gate confirms that the evaluation is internally consistent.

only the per-recording statistics can. This is our principal mitigation against prompt-knowledge contamination (Section 9).

Reasoning beyond the detection limit—one solved, one honestly

capped. IDV3 and IDV4 lie beyond classical PCA monitoring and are excluded by FaultExplainer’s PCA implementation [8, 9, 1]. IDD resolves IDV4 to DETERMINED (cooling-loop heat-removal step, confidence 0.75) purely from the control-response signature. IDV3—by common consent the hardest TEP fault, with a deliberately weak signature—terminates in a capped COMPETING_SET (0.60): the exclusion chain eliminates four mechanism families (reactor thermal imbalance, stripper-local, utility, instrumentation artifacts) and localises the disturbance to the feed-composition balance (coherent pressure rise across all three vessels with A deficit/C enrichment in the purge), but the inlet site cannot be named because streams 1–4 carry no composition analyser. The verdict names exactly the missing measurement. We report this as what it is—a mechanism-level miss on the hardest fault—and note that the alternative (forcing a single cause from indistinguishable evidence) is the failure mode the protocol exists to prevent.

Blind-set robustness. The SKAB cavitation scenario is drawn from the “other” experiment group whose fault label is withheld from the scenario description. The pipeline’s top-ranked surviving hypothesis is the cavitation-type two-phase instability (three intermittent flow-collapse windows totalling 76 near-zero-flow seconds with same-window vibration and fluid-temperature excitation, and the drive side excluded), but pressure-channel quantization (0.328 bar steps) masks the closed-valve head signature that would separate cavitation from intermittent passage blockage, so the verdict is a capped COMPETING_SET (0.58) naming the missing high-resolution pressure channel.

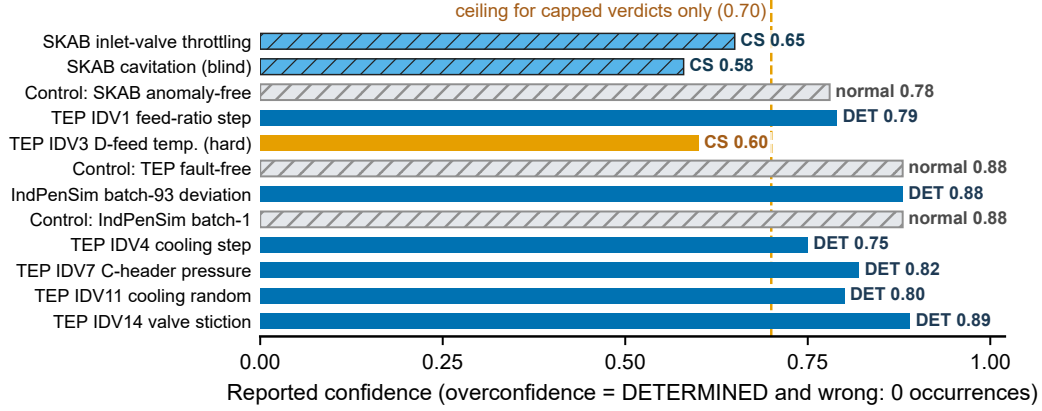


Figure 4: Reported confidence per scenario in the full-pipeline benchmark run. Solid dark blue bars are DETERMINED fault verdicts; hatched sky-blue bars are capped COMPETING_SET verdicts whose survivors rank the true mechanism (SKAB valve 0.65, SKAB cavitation 0.58); the solid amber bar is the single capped verdict whose survivors exclude the true mechanism (IDV3, 0.60); hatched grey bars are the three normal-operation controls; the dashed line is the 0.70 COMPETING_SET ceiling. Each capped verdict names the discriminating measurement that would separate its surviving mechanisms; no conclusion exceeded the ceiling implied by its evidence levels (overconfidence 0).

7.3. Ceiling Compliance and False-Alarm Suppression

Fig. 4 plots the reported confidence per scenario. Fault-scenario confidences span 0.58–0.89; the three COMPETING_SET verdicts sit in the 0.58–0.65 band because the protocol caps them, and each names its discriminating measurement (valve-position/command channel; high-resolution pressure; stream-1–4 composition analysers). We report this metric as *ceiling compliance* rather than calibration, and we state its circularity plainly: the gate enforces the ceiling, so compliance is a property of the protocol’s arithmetic, not an empirical discovery. Worked examples: IDV4’s DETERMINED verdict at 0.75 additionally carries the reason-coded DIRECT_MEASUREMENT

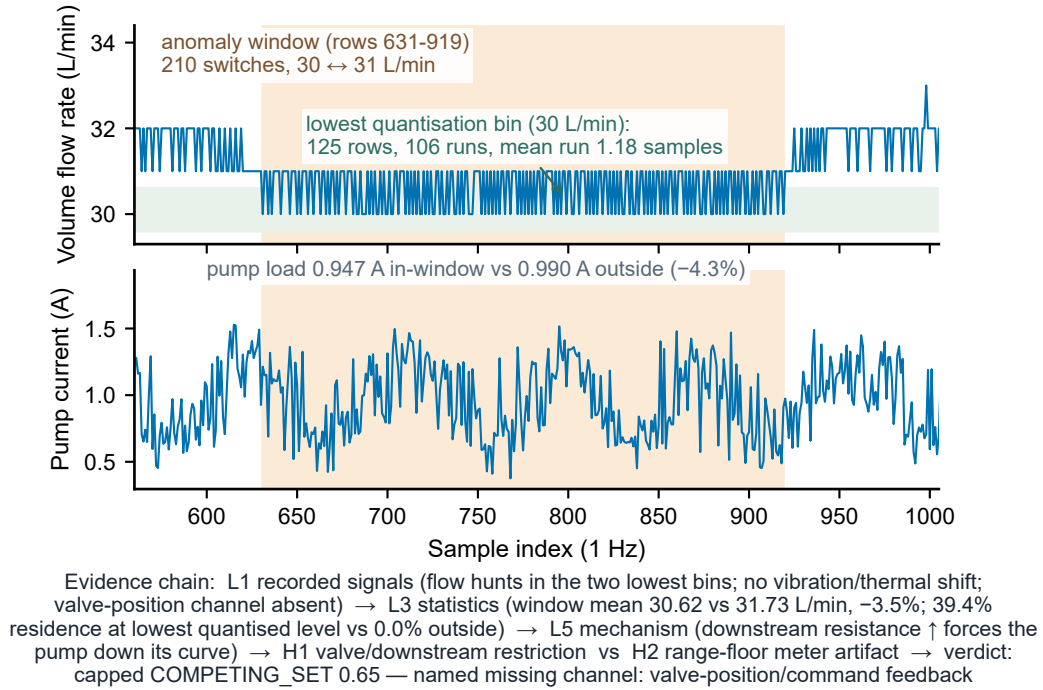


Figure 5: Worked example of an honestly capped verdict (SKAB inlet-valve throttling, the benchmark’s single verdict-type compliance miss). Top: the recorded flow signal hunts between the two lowest quantisation bins (30/31 L/min; 210 switches; pale-amber shading marks the fault window and the pale-green band marks the floor bin) while the pump current barely moves (-4.3%). The shaded window (rows 631–919, 289 samples) contains 125 samples at the 30.0 L/min quantisation level—the in-panel annotation’s row count and its 106 runs—whereas the pipeline’s separate floor-residence metric reports 39.4% of the window; the two measures are named and reported separately, not equated. Bottom strip: the evidence chain as the pipeline recorded it, from the raw signal through L3 statistics and the L5 mechanism to the two surviving hypotheses and the capped verdict with the named discriminating measurement. Every annotation is read from the released run artifacts.

ceiling noted in Section 4; the SKAB cavitation COMPETING_SET at 0.58 sits under the 0.70 capped-verdict ceiling. A psychometric calibration study (pre-gate confidence versus outcome correctness, over repeat runs) remains

future work. Under the benchmark’s strict typing, one scenario counts as a verdict-type miss: SKAB inlet-valve throttling returns `COMPETING_SET` where the scenario’s allowed set admits only `DETERMINED` (the ground truth is known to be valve restriction); the typing is one-directional — the other eight fault scenarios’ allowed sets admit both states, so compliance can flag over-capping but never under-capping. We report that miss honestly—and note that the capped verdict is the act the protocol demands when the valve-position and controller-command channels that would separate “valve closing” from “commanded duty reduction” are absent from the recording; the pump’s electrical load falls only 4.3% ($0.990 \rightarrow 0.947$ A, within the flicker background) while flow collapses into the lowest quantisation bin—a duty-share signal too small to resolve the question. Fig. 5 makes this scenario fully visible: the annotated recording (flow hunting between the two lowest quantisation bins with the pump load essentially flat) alongside the evidence chain as the pipeline recorded it—raw signal, statistics, mechanism, the two surviving hypotheses, and the named missing channel. Overconfidence—`DETERMINED` *and* wrong—never occurs (under the calibrated keyword set; Section 5).

Control scenarios demonstrate the complementary discipline: the normal IndPenSim batch contains recipe-dosing impulses reaching $|z|=16.2$, which the pipeline attributes to recipe steps and closed-loop control actions (one judge-repair round was spent sharpening exactly this attribution) rather than to faults; the normal TEP run exhibits control-loop correlations at $|r|=1.0$, which the anti-spurious filter classifies as definitional. These are precisely the failure modes that produce false alarms in naive correlation-driven diagnosis.

Table 7: Per-Scenario Results With the Decisive Statistical Signature (abbreviated; full chains in the released grading records). Verdict column: DET = DETERMINED, CS = capped COMPETING_SET, with reported confidence. Rubric deductions: SKAB inlet-valve throttling $-15 = R4$ verdict-type compliance; TEP IDV11 $-10 = R6$ physics-check coverage. Judge scores are post-threshold (≥ 90) and carry no benchmark weight.

Scenario (mechanism)	Verdict	Rubric	Decisive signature
SKAB inlet-valve throttling	CS 0.65	85	flow hunting between the two lowest bins (30.62 vs 31.73 L/min, -3.5% ; 39.4% floor dwell); pump load -4.3%
SKAB cavitation (blind set)	CS 0.58	100	intermittent flow collapse (76 rows < 10 L/min); same-window vibration $+5.9\%$; cavitation-type instability top-ranked
TEP IDV1 A/C feed-ratio step	DET 0.79	100	A-feed valve $+204\%$ ($z=126$); A+C feed -5.9% ; steam $+29.7\%$
TEP IDV3 D-feed temp. (hard)	CS 0.60	100	composition-balance shift (vessel pressures $+0.71$ – 0.92σ coherent); inlet site unresolvable (no analysers)
TEP IDV4 cooling (step)	DET 0.75	100	XMV_10 step $+6.8\sigma$ plateau (sole $> 1.5\sigma$ column); reactor temp $z=10.06$, re-controlled
TEP IDV7 C-header pressure	DET 0.82	100	feed flow -11.9σ ; XMV_4 $+25.3\%$ holds setpoint
TEP IDV11 cooling (random)	DET 0.80	90	variance inflation VR = 16.45 with $\Delta\text{mean} = 0.006\sigma$; no step, no stiction
TEP IDV14 valve stiction	DET 0.89	100	80 events $>5\sigma$ (peak 25.3σ), ≈ 30 min period, no net drift; cooling side leads temp 15 min ($r=-0.97$)
IndPenSim batch-93	DET 0.88	100	CW loop clamped (≈ 2.0 vs 29.8 L/h) $14.6\text{ h} \rightarrow +3.3\text{ K}$; recovery $\rightarrow$ in band $\leq 2\text{ h}$
Controls (3)	3 DET (normal)	100 (all)	normal-operation verdicts at confidence 0.78–0.88; zero false alarms

8. Comparison With Published Baselines, Ablation, Stability, and a Case Study

8.1. Register A: Per-Fault Comparison on TEP

FaultExplainer [1] reports, in its root-causes-included regime, root-cause correctness of 7/11 (63.6%, Wilson CI [35.4%, 84.8%]) for GPT-4o and 9/11 (81.8%, CI [52.3%, 94.9%]) for o1-preview on the 11 PCA-detectable TEP faults—where correctness means the true fault or an alias appears in the model’s top-3 list, with candidates supplied in the prompt. In its own no-candidate regime (General-Reasoning prompt), both models place the correct or a related cause in the top-3 for 8/11 faults under lenient grading. IDD, without candidate lists and with a single strict mechanism-keyword verdict, is Top-1 correct on **5/6** faults of its TEP subset (Wilson CI [43.6%, 97.0%]; 4/6 under the pre-revision keyword set, Section 5); on the four shared PCA-detectable faults it is correct on 4/4, where the GPT-4o baseline is 3/4 (IDV1 mis-diagnosed) and o1-preview is 4/4 under the alias-inclusive scoring. On the two PCA-excluded faults that both baselines leave unscored, IDD resolves IDV4 and honestly caps IDV3. Three caveats bound this comparison: the base models differ and are not controlled; FE’s lenient top-3-with-alias protocol is not directly comparable to a strict single-verdict metric; and the model-controlled ablation of Section 8.3 shows that a bare call to the same model family already scores these documented faults at or above the pipeline (9/9 versus 6/9 resolved)—so we read the per-fault table descriptively, not as a superiority claim. Table A1 (Appendix A) transcribes the same comparison in machine-readable form. Fig. 6 gives the per-fault breakdown against the transcribed FaultExplainer Table 1 outcomes and PCA detectability; Fig. 7

visualises the aggregate point estimates with their intervals. Three per-fault outcomes stand out:

- **IDV1 (A/C feed-ratio step):** the GPT-4o baseline mis-diagnoses this fault (o1-preview resolves it); IDD resolves it to DETERMINED (0.79) from the actuator-flow signature (A-feed valve step +204%, $z=126$, with the A-feed flow +204% at $z=127$ against a -5.9% composite-feed change), correctly restructuring the disturbance as an A:C supply-weight shift with plant-wide pressure and steam-compensation consequences.
- **IDV3 and IDV4 (excluded from FE scoring):** both faults are excluded from FaultExplainer’s scoring by its PCA implementation (classical detectability: IDV3 at the limit, IDV4 weakly detected). IDD resolves IDV4 (DETERMINED 0.75, cooling-loop heat-removal step) and terminates IDV3 in a capped COMPETING_SET (0.60) that eliminates four mechanism families and localises the disturbance to the feed-composition balance—a partial result on the hardest fault, reported as such rather than forced.
- **IDV7, IDV11, IDV14:** both systems are correct; IDD additionally separates the three cooling-water perturbation *forms*—step, random variation, and stiction—at the level of distinct quantitative signatures (Section 7), a granularity below fault-identity level.

Because the scenario sets, prompt regimes, and grading protocols all differ (top-3 with alias acceptance vs. a strict single verdict; with vs. without candidate lists; different base models), we refrain from a formal significance test

	FaultExplainer GPT-4o	FaultExplainer o1-preview	PCA detectable	IDD (this work) no candidates
IDV1 A/C feed-ratio step	wrong	correct	yes	Top-1 (0.79)
IDV3 D-feed temperature step (hard)	unscored	unscored	no	capped CS (0.60)
IDV4 reactor cooling-water step	unscored	unscored	no	Top-1 (0.75)
IDV7 C-header pressure loss	correct	correct	yes	Top-1 (0.82)
IDV11 cooling-water random	correct	correct	yes	Top-1 (0.80)
IDV14 cooling-valve stiction	correct	correct	yes	Top-1 (0.89)

Figure 6: Per-fault comparison on the TEP subset: IDD vs. FaultExplainer (transcribed Table 1 outcomes, arXiv:2412.14492; top-3 with alias acceptance) and PCA detectability as used in FaultExplainer’s scoring (classical detectability: Russell et al. [8], Chiang et al. [9]). IDD is Top-1 correct on 5/6 without candidate lists (Section 5), including IDV1 (mis-diagnosed by the GPT-4o baseline) and the PCA-excluded IDV4; IDV3 ends in a capped COMPETING_SET with the discriminating measurement named. Cell colours: dark blue = IDD Top-1-correct; amber = IDD capped COMPETING_SET; green = FE-correct; vermillion = FE-wrong; pale grey = FE-unscored cells and PCA-detectable “yes”; grey “no” = PCA-excluded.

and present the per-fault table as descriptive evidence. On the four matched faults IDD is correct wherever either baseline is correct (IDD 4/4; GPT-4o 3/4; o1-preview 4/4); it resolves the GPT-4o baseline’s one miss (IDV1); and on the unscored PCA-excluded pair it delivers one resolved diagnosis and one honestly capped verdict with named discriminating data. The comparison is also asymmetric in cost: FE-style prompting runs in minutes per fault, whereas the full gated pipeline’s per-scenario agent time is one to nearly four hours (Section 6).

8.2. Register B: Reference-Only Context

For context we report sample-level results from the literature without ranking: TEP classification accuracies of 0.887–0.907 (MLP/GRU/TCN on

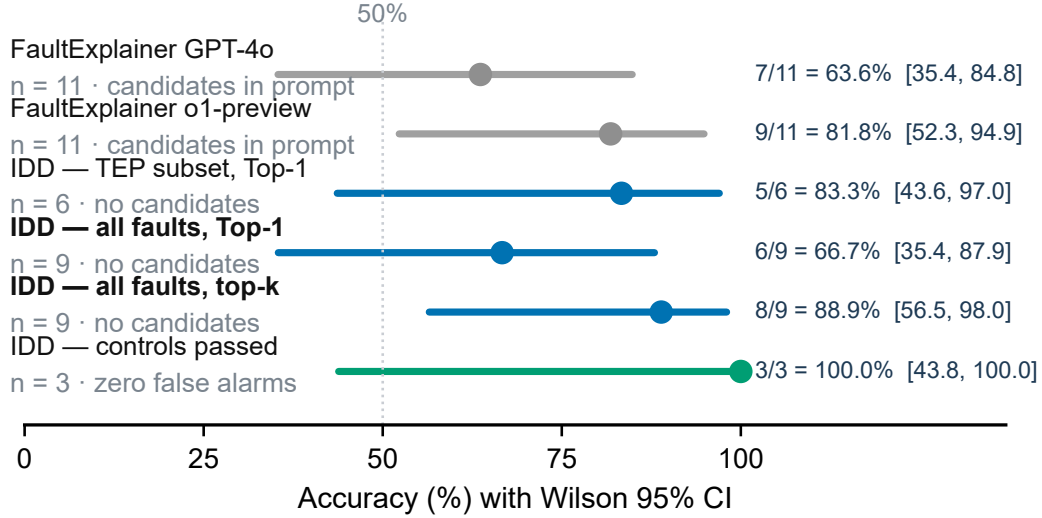


Figure 7: Register-A aggregate comparison with Wilson 95% intervals ($z = 1.96$). Grey: FaultExplainer-reported single-LLM baselines ($n=11$, candidates in prompt). Blue: IDD full-pipeline execution without candidate lists—the TEP subset (5/6), all nine faults (6/9), and top-k (8/9). Green: the three controls (3/3, no false alarms). Interval labels for rows at 100% extend past the axis. Sets differ, so overlapping intervals are expected and no significance test is claimed; the per-fault breakdown (Fig. 6; Table A1) is the finer-grained evidence (5/9 pre-revision; TEP subset 4/6; Section 5).

the 28-class extended TEP taxonomy of Rieth et al. [6] [5]), anomaly-detection F1 of 0.970 (BeatGAN [13]), SKAB ensemble F1/AUC of 0.85/0.88 [40], and leakage-controlled bearing AUROC of 0.75–0.93 [14]. These systems answer “is this sample anomalous / which class”; IDD answers “what is the mechanism, how confident, with which evidence”. The dimensions are complementary: a deployment would plausibly chain a sample-level detector (first-stage alarm) with a scenario-level RCA system (this work).

8.3. Model-Controlled Baselines and Pipeline Ablation

Because the base model is a confound in any comparison with published LLM baselines (Section 6), we ran three model-controlled baselines with the *same model family and deployment* as the pipeline (Table 8; the LLM arms are recorded replays of live runs from this deployment, and the TEP fault-free control was not run bare): (i) a classical PCA baseline—normal-control training, T^2/Q detection with 99th-percentile alarms, and SPE-contribution diagnostics [9, 7]—on all twelve scenarios; (ii) a replication of FaultExplainer’s protocol on the six TEP faults, in both of FE’s prompt regimes, scored both FE-style (top-3 with alias classes) and strictly; and (iii) the pre-registered single-LLM ablation—one call to the same model family over the identical blinded statistical digest, no ontology, no pipeline—on all nine fault scenarios, scored with the same strict mechanism keywords as IDD. All three baselines are bare single-call executions: no agent orchestration, no gates, no evidence grading—and, like the pipeline runs, each is a single recorded execution rather than a repeated sample. The classical PCA arm is computed by a released deterministic script (pure-Jacobi eigendecomposition, normal-control training, 99th-percentile alarms of the control distribution, TEP fault-window evaluation from row 161); an earlier prototype run agreed with it on every SPE statistic and control rate, with minor T^2 differences on the weakest-signature faults, and the prototype output is archived in the released repository for audit.

Three findings emerge. **First**, on keyword-scored accuracy over documented public faults the bare same-model call is not merely equal to the full pipeline but ahead of it: 9/9 strict hits versus the pipeline’s 6/9 resolved

Table 8: Model-Controlled Baselines on the Benchmark (same model family and deployment as the IDD pipeline; single call per scenario; answers in the released repository). LLM arms are recorded replays of live runs; the PCA and FE-protocol arms are deterministic re-executions.

Baseline			Candidates	Scoring		Scope	Result		
Classical control training)	PCA	(normal-	—	detection rate		12 scen.	IDV3	T^2	2.9%/SPE
							4.8%;	IDV4	SPE 100%
							vs T^2	48.8%;	controls 1%
Classical PCA (diagnosis)			—	SPE contribution		12 scen.	top-3 variables only; no mechanism verdict		
FE-protocol (GLM)	replication		yes	FE-style	top-3 + alias	TEP 6 faults	6/6		
FE-protocol (GLM)	replication		yes	strict	single-verdict	TEP 6 faults	6/6		
FE-protocol (GLM)	replication		no	strict	single-verdict	TEP 6 faults	6/6		
FE official code (GLM), T2-contrib features			yes	FE-style	top-3 + alias	TEP 6 faults	5/6		
FE official code (GLM), T2-contrib features			yes	strict	single-verdict	TEP 6 faults	5/6		
Single-LLM, identical digest (GLM)			no	strict	single-verdict	9 faults	9/9		
Single-LLM, controls (GLM)			no	normal verdict		2 controls	2/2, zero false alarms		
IDD full pipeline			no	strict	resolved	9 faults	6/9	(3 capped	COMPETING_SET)
				Top-1					

(three capped verdicts)—including IDV3, plausibly by literature recall of the documented weak-signature fault, and naming valve and cavitation mechanisms on the SKAB scenarios from their signatures—and it produces no false alarms on the two of the three controls the bare-LLM arm covered (the TEP fault-free control was not run bare; the released baseline record notes the coverage gap, and only covered controls are scored).

Second, the FE official code, running on the same GLM model, also scores 5/6—identical to IDD’s TEP accuracy but with a different miss pattern: FE misses IDV4 because its PCA T^2 features are dominated by feed-composition shifts that mask the cooling-water signal, while IDD misses IDV3 because it honestly caps a `COMPETING_SET` where the bare LLM forces IDV3 (correctly, by literature recall).

Third, the classical PCA baseline reproduces the literature (IDV3 at the detection limit, T^2 2.9%/SPE 4.8%; IDV4 “weak T^2 , strong Q ”—SPE 100% vs T^2 48.8%) and produces no mechanism verdicts.

The three systems fail in three different ways, revealing that benchmark accuracy on documented faults conflates detection ability, mechanism attribution, and epistemic discipline. What the bare call does not produce is anything the audit plane can verify: no evidence levels, no ceiling compliance, no physics check, no execution proof, no artifact provenance—and its IDV3 answer, being correct, is exactly the kind of unverifiable guess that the three-state protocol exists to constrain (the pipeline’s capped verdict names the stream-1–4 composition analysers that would settle the question).

The classical PCA baseline quantifies the detection–diagnosis gap directly: IDV3 sits at the detection limit (2.9% T^2 , 4.8% SPE), IDV4 is detected reliably only by the SPE statistic (100% vs 48.8%), and the output stops at variable contributions—no mechanism, no confidence, no verdict state. Attribution of the pipeline’s remaining accuracy margins (e.g., whether the anti-spurious filter or the ontology reuse contributes) still requires the filter-disabled and engine-substitution arms, which remain future work. A reproducibility note completes the arm: the deterministic suite

arms (classical PCA, FE-protocol feature extraction, prompt assembly) were re-executed after the benchmark closed and reproduced all 45 output files byte-for-byte with timestamps excluded (released verdict: DETERMINISTIC; the re-execution covers the deterministic suite arms, not the stochastic reasoning layer), so the baseline numbers are exactly as reproducible as the deterministic layer they run on. Fig. 8 arranges the complete comparison as a per-scenario matrix: the pipeline’s verdict column against the two LLM-baseline arms and the three classical-detection statistics, making visible both where the systems agree (all nine faults are keyword-hit by the bare LLM, and the covered controls pass—two of three; released baseline record) and where they differ in kind (the pipeline’s three capped verdicts and the classical detectors’ blindness to IDV3 and to IDV4’s T^2 channel).

8.4. Verdict Stability Across Independent Runs

To test whether the pipeline’s conclusions are stable across independent agent sessions, three scenarios spanning the three verdict archetypes were re-executed through the full gated pipeline in fresh agent sessions (new ontology build, new diagnosis, new review cycle) on identical prepared data; a fourth scenario (TEP IDV11) was subsequently re-executed by the random-draw protocol described below, and its outcome bounds the stability claim. All three scenarios returned identical verdict types across independent sessions (3/3 type consistency; Table 9). The two DETERMINED verdicts identified the same root-cause mechanisms in both rounds (IDV1 A/C feed-ratio step; IDV14 valve stiction), with confidence varying by ± 0.07 – 0.09 —within the expected range for stochastic LLM reasoning under a fixed protocol. A signature-level caveat applies to IDV14: both sessions agreed on the mecha-

Table 9: Verdict Stability Across Independent Pipeline Executions (same data, fresh agent sessions). Session dates from the released run-directory names: IDV14 R1 = 2026-09-14 (202609141855391), R2 = 2026-09-15 (202609150454479); the ≈ 12 min vs ≈ 30 min period discrepancy of Section 8.4 maps onto this revision boundary between the two sessions. For TEP IDV1, R2 is the scenario’s canonical graded run—the pair reads the superseded earlier run against the canonical run.

Scenario	Verdict R1	Verdict R2	Type Match	Root Cause	ΔConf
TEP IDV1 A/C feed-ratio step	DET 0.88	DET 0.79	✓	IDV1 (both runs)	−0.09
TEP IDV14 valve stiction	DET 0.82	DET 0.89	✓	IDV14 (both runs)	+0.07
SKAB inlet-valve throttling	CS 0.65	CS 0.65	✓	capped (both runs)	0.00

nism, the affected loop, and the 15 min cooling-side lead, but their quantitative characterisation of the oscillation differs—the earlier session reported an ≈ 12 min period with a period-4 autocorrelation fingerprint across the three coupled channels, the later an ≈ 30 min median period over $80 > 5\sigma$ events with no such fingerprint. Verdict type and mechanism are therefore stable across sessions while the fine-grained period estimate is not. Every row of Table 7 is read from the single execution named in that scenario’s grading record—for IDV14 the later (canonical, graded) run—and the earlier run is retained in the released corpus. We bound the claim this way deliberately: a signature column summarising one execution should not be read as a reproducible period measurement. The `COMPETING_SET` scenario returned the same capped verdict with identical ceiling in both rounds. Beyond this targeted check, the consistency study is formalised as an automated evaluation stage (`run-tier stability`, released with the evaluation) that re-scans every recorded independent execution of every scenario—41 run directories at the time of writing—using the same extraction semantics as the scorer,

and reports agreement *within system versions*: across the recorded corpus, within-version verdict-type and Top-1 agreement is 15/16 case-version pairs (case-version pairs: scenario–version combinations with at least two recorded runs; pre-discipline era 12/12; current era 3/4); the early-era pairs are agreement measurements over provenance-verified run directories of the earlier pipeline version, and the archived pre-benchmark note era contributes no pairs. Fig. 9 maps the complete audit per scenario: every proven execution at its reported confidence, era-coded, with the audit status and the four re-tested current-era pairs. Three disclosures bound this claim. First, the consistency protocol includes a *random* re-test draw—a scenario is selected by a seeded uniform draw over the nine-fault pool, so the re-tested scenario cannot be hand-picked after seeing results; the draw record is replayable from the released seeds. Five seeded draws are recorded: the first selected TEP IDV3, the next three all selected TEP IDV11, and the fifth (seed 368776031, $u = 0.131221641$) selected SKAB cavitation; both drawn-but-unexecuted retests (TEP IDV3 and SKAB cavitation) remain outstanding execution contracts, as do second executions for the remaining scenarios—we report the gaps rather than substituting hand-picked scenarios. Second, the thrice-drawn IDV11 contract has been fulfilled by a fresh full-pipeline execution (2026-09-17), and its outcome is the corpus’s one *within-era divergence*: the canonical session resolved IDV11 to DETERMINED at 0.80, the drawn re-execution returned a capped COMPETING_SET at 0.62 that separates cooling-water inlet-temperature fluctuation (top-ranked, evidence-favoured) from actuator jitter. The audit’s type-first rule therefore marks the pair DIVERGENT, although the pair agrees on the implicated actuator

(XMV_10), the mechanism class, and the top-ranked mechanism, and both verdicts are protocol-conformant—the flip is in the verdict *state*, not in the localisation. We read this as the honest boundary of the stability claim: verdict *state* on IDV11 is session-sensitive (the canonical run’s single-survivor call and the retest’s two-survivor cap sit on either side of a genuinely fine discrimination), which is precisely why the $k \geq 3$ repeat study below is pre-registered rather than assumed. Third, on the remaining current-era pairs the audit finds agreement of *two consistent and one weak* signature match: the weak pair is SKAB inlet-valve throttling, whose two executions share the verdict type and the implicated channel but phrase the surviving-hypothesis ordering differently—the same scenario that carries the benchmark’s single verdict-type compliance miss. Verdict-level and mechanism-level stability are therefore reported as distinct measurements rather than collapsed into one number. The three sensitive scenarios (SKAB valve, SKAB cavitation, TEP IDV3) flip from uncapped DETERMINED in the early era to capped COMPETING_SET in the current one—a documented tightening of the pipeline’s epistemic discipline between versions, not run-to-run instability (the tightening also rewrote the TEP briefs after an internal audit found ground-truth phrasing in five of the six, and replaced the benchmark’s degenerate self-declared judge score—one note value copied across ten criteria—with a disclosed provenance label plus an independent structural audit; the audit record is released as `docs/benchmark/AUDIT-2026-09-13.md`); within each version the verdicts agree except for the disclosed IDV11 state flip. Expanding the current-era repeat count (all 12 scenarios, $k \geq 3$ repeats, verdict-state stability and confidence distributions) remains pre-registered in

the released repository.

8.5. Case Study: An Auditable Evidence Chain on a Fault Excluded from Classical Scoring

For TEP IDV4—a fault excluded from FaultExplainer’s scoring by its PCA implementation, and only weakly detected classically [8]—the recorded chain reads: (i) *L3 statistical evidence*: the cooling-valve XMV_10 steps from 40.7% to 47.3% at the fault onset and holds a $+6.8\sigma$ sustained offset—the sole column exceeding 1.5σ among all 52—while reactor temperature spikes to $z=10.06$ and is then re-controlled; (ii) *L5 mechanism*: a step degradation in cooling-loop heat removal (inlet-temperature rise or exchanger fouling) forces the controller to open the valve to hold temperature, so a sustained valve offset with re-controlled temperature is precisely the control signature of heat-removal degradation, not of a sensor artifact; (iii) *eliminations*: feed, purge, and condenser alternatives stay within 1.3σ throughout the fault window; (iv) *conclusion*: DETERMINED, confidence 0.75, with the sub-attribution between “cooling-water inlet-temperature step” and “heat-exchanger degradation” honestly flagged indistinguishable—the recording contains no cooling-water inlet measurement, and the verdict names it. The full chain, together with the physics-check verdict, the judge’s ten-criterion scores (92/100), and the independent HTML review (pass), is persisted in the grading record and released with the evaluation.

SKAB inlet-valve throttling	CS 0.65 ranked	hit	-	100	100	-
SKAB cavitation (blind)	CS 0.58 ranked	hit	-	100	100	-
Control: SKAB anomaly-free	normal 0.78	normal	-	1	1	-
TEP IDV1 feed-ratio step	Top-1 0.79	hit	hit	99	100	det.
TEP IDV3 D-feed temp. (hard)	CS 0.60 unranked	hit	hit	3	5	no
Control: TEP fault-free	normal 0.88	n/a	-	1	1	no (correct: control)
IndPenSim batch-93 deviation	Top-1 0.88	hit	-	15	74	-
Control: IndPenSim batch-1	normal 0.88	normal	-	1	1	-
TEP IDV4 cooling step	Top-1 0.75	hit	hit	49	100	det.
TEP IDV7 C-header pressure	Top-1 0.82	hit	hit	100	100	det.
TEP IDV11 cooling random	Top-1 0.80	hit	hit	54	66	det.
TEP IDV14 valve stiction	Top-1 0.89	hit	hit	100	93	det.
	IDD verdict (conf.)	Bare LLM (strict)	FE style (cand.)	PCA T^2 det. %	PCA SPE det. %	FE prot. det.
	■ IDD resolved (Top-1)	■ capped CS, mechanism ranked	■ miss / not detected			
	■ baseline hit / control pass	■ capped CS, not ranked	■ not in protocol scope			

Figure 8: Per-scenario comparison matrix across the model-controlled baseline suite (all arms executed on the same prepared recordings; every cell is read from the released run artifacts). Column 1: the pipeline’s recorded verdict with its reported confidence—dark blue resolved DETERMINED (Top-1), hatched sky capped COMPETING_SET with the true mechanism still ranked among the surviving hypotheses, amber capped COMPETING_SET without the true mechanism ranked, green “normal” for controls. Column 2: bare single-call LLM under strict mechanism keywords (no candidate list; the bare-LLM arm’s n/a for the TEP control is an execution-coverage gap, not a pass). Column 3: the same model under FaultExplainer’s candidate-list regime, scored FE-style (top-3 with alias classes; “–” = regime not applicable outside TEP). Columns 4–5: classical PCA detection rates for T^2 and SPE, drawn on an independent sequential ramp (darker = higher) that does not reuse the semantic colours of columns 1–3. Column 6: whether FaultExplainer’s six-consecutive- T^2 trigger fires on the recording for fault scenarios (grey “no” for the fault-free control is the correct outcome; “–” on the non-TEP fault rows marks that this detection arm was not run outside TEP)—a detection result, distinct from the scored FE-official answers of Table 8.

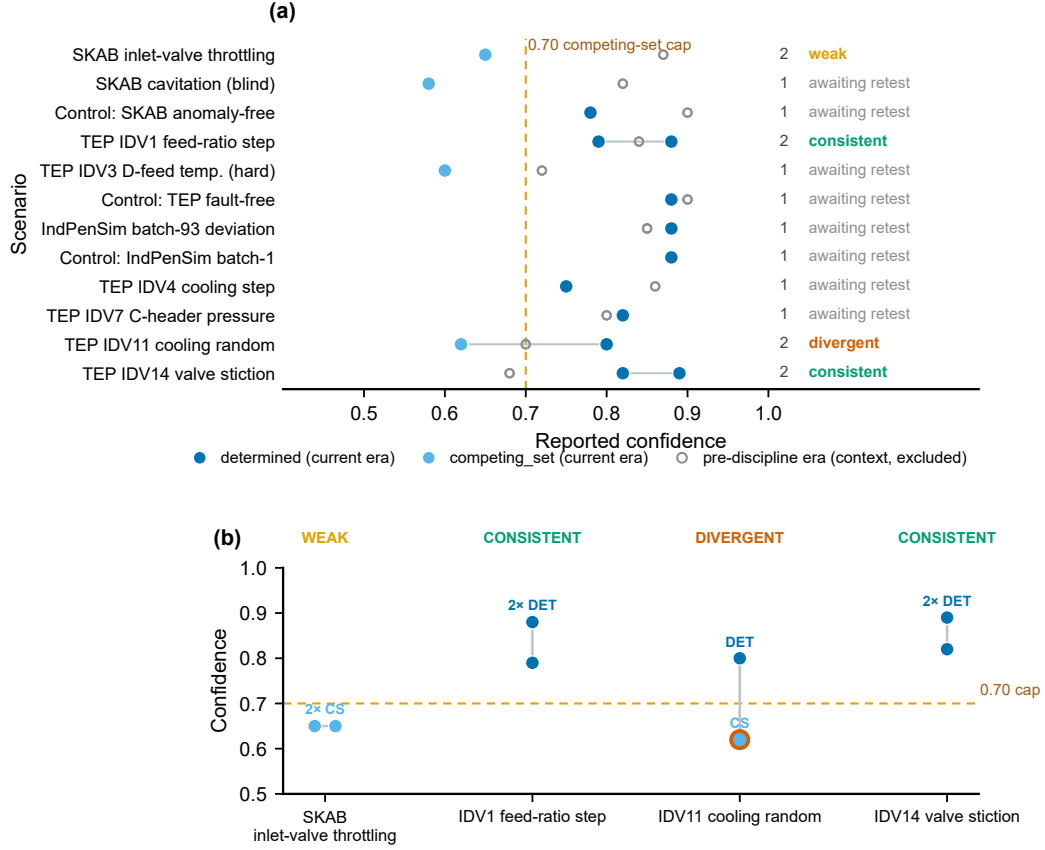


Figure 9: Consistency audit of every recorded independent execution, read from the released audit artifact. (a) Each marker is one proven execution placed at its reported confidence: filled markers are current-era runs (dark blue DETERMINED, sky COMPETING_SET), grey open markers are pre-discipline-era runs shown as context only and excluded from the headline (the era boundary is a documented system revision, Section 8.4); the right-hand columns give the number of current-era proven runs and the audit status; eight scenarios currently hold a single current-era execution and carry an explicit re-run contract rather than an assumed stability claim. (b) The four re-tested current-era pairs with per-run verdict type; the single within-era divergence is TEP IDV11, whose drawn re-execution (vermillion ring) returned a capped COMPETING_SET at 0.62 where the canonical session resolved DETERMINED at 0.80—same actuator (XMV_10), same mechanism class, true mechanism top-ranked in both. Dashed line: the 0.70 COMPETING_SET confidence cap.

9. Discussion, Limitations, and Deployment Prospects

Statistical power and single-execution evidence. With $n=9$ fault scenarios, Top-1 = 66.7% (5/9 under the pre-revision keyword set) supports only a wide interval ([35.4%, 87.9%] at 95% confidence). Two further qualifications matter. First, every headline number is a *single recorded execution*: multi-agent LLM pipelines are stochastic, so each point estimate is one draw from an uncharacterised run-to-run distribution. Second, the bounded repair loop introduces a selection effect—a reported verdict may be the product of up to five attempts—although in this corpus all but one scenario closed with zero repairs, the exception being a single repair round on one control (repair counts in the released event logs; Section 6). The per-fault evidence remains more informative than the aggregate: six of nine faults map onto published per-fault outcomes, and on matched faults IDD is correct wherever either baseline is correct. The model-controlled ablation (Section 8.3) already bounds the accuracy claim (9/9 bare versus 6/9 resolved)—so the pipeline’s value on documented faults is process quality, not accuracy. The repeat study is formalised as an automated evaluation stage (era-aware within-version agreement 15/16 case-version pairs across all recorded executions—a preliminary figure, with eight of twelve scenarios awaiting a second in-era execution—and the single within-era divergence, a drawn retest flipping IDV11’s verdict state while agreeing on localisation, mechanism class, and the top-ranked mechanism, disclosed in Section 8.4); expanding the current-era repeats (all 12 scenarios, $k \geq 3$: verdict-type stability, confidence spread, repair counts) remains pre-registered and partially executed; extending the corpus (the full 20-class TEP catalogue with the

extended simulation dataset [41]; bearing datasets; novel-disturbance suites) is the designed path to tighter intervals.

Prompt-knowledge contamination. TEP’s documented fault mechanisms appear in textbooks and are therefore plausibly memorised by frontier models. Our protocol mitigates this at the level where memorisation actually operates—but the mitigation has a limit we state plainly. The documented cause table records not only the disturbed channel but each fault’s dynamic form (IDV4 step, IDV11 random variation, IDV14 stiction), so recall of that table alone could reproduce the observed mechanism labels; the statistics-driven reading rests on the recorded reasoning chains, each of which cites the per-recording statistic that discriminates its form. The SKAB “other”-group scenario removes the fault label from all natural-language brief content—the one residual channel we disclose is the scenario identifier itself, whose case id embeds the English fault word—and still yielded a correctly top-ranked (though honestly capped) cavitation-type mechanism. The RAG channel’s engagement varied run by run (per-run detail in Section 3.2), yet no L6-tagged item enters any surviving chain as decision evidence—the single item carrying the combined L5/L6 external marker in the released corpus is marked corroboration-only, so no verdict in the benchmark corpus rests on external evidence. A novel-disturbance suite (combination perturbations absent from the literature) is the definitive control and remains under construction.

What the three capped verdicts mean. The three `COMPETING_SET` outcomes are the protocol working as designed, and they are also the benchmark’s most transferable finding: in each case the pipeline (i) localised

the disturbance to a subsystem, (ii) eliminated whole mechanism families with quantitative contradictions, (iii) ranked the true mechanism first in two of three cases, and (iv) named the single measurement—valve-position/command channel, high-resolution pressure, upstream composition analysis—whose addition would collapse the set to DETERMINED. In deployment terms these verdicts are work orders for targeted instrumentation, not failures. To keep the framing falsifiable, the released scenario definitions record the expected verdict set of every scenario, fixed before execution; SKAB inlet-valve throttling is the one scenario whose expectation was DETERMINED-only, and its capped verdict is counted as the benchmark’s single compliance miss rather than narrated as a success.

Inference authorship and provenance. Every scored artifact in this benchmark is authored by a gated pipeline stage—the diagnosis by the diagnostician, the quality verdict by the judge, the ENDORSED decision by the physical auditor, the review verdict by the independent HTML reviewer—and each carries a provenance label in the grading record. Because self-declared quality scores are not verifiable, the benchmark’s reported quality metric is the deterministic compliance rubric (Section 6)—and we disclose that the rubric, the grading keywords, and the allowed verdict sets are author-designed instruments measuring conformance to this paper’s own protocol; their release with the repository makes them auditable, but independent anchoring (the blinded expert adjudication of the released evidence chains) is the outstanding validation step. The LLM ten-criterion judge remains the pipeline’s internal gate, and its scores are reported as execution-quality evidence only. An internal integrity audit that found (and removed)

a scripted shortcut capable of fabricating artifacts and agent events is itself part of the released history; the completeness gate now makes that shortcut unreplayable. What is still missing is a comparison against *human* experts on identical digests; an expert-vs-pipeline blind test is planned. The single-LLM ablation of Section 8.3 also bounds what the pipeline can claim: on documented public faults, a bare call to the same model family is already accurate—so the pipeline’s justification is the audit, compliance, and calibrated-uncertainty machinery, and on novel disturbances (where recall cannot substitute for reasoning) the comparison must be re-run.

Cost. A full nine-stage pipeline execution spends one to nearly four hours of agent time per scenario (56–220 minutes across the twelve canonical runs; median ≈ 110)—orders of magnitude more expensive than a fitted classifier at inference time, and far slower than single-prompt baselines. The economic comparison is only meaningful against the *engineering cost of obtaining root causes* (expert hours), not against sample-level inference latency; and the deterministic statistical stage, which carries most of the computation, is script-speed.

9.1. Industrial Deployment Prospects

We separate what the benchmark and one operational deployment *demonstrate* from what the architecture *projects*.

Observed in one deployment (artifacts anonymised and not part of the released corpus): a film-line run exercised the pipeline end to end on episodic scratch defects of a biaxially-oriented polyester film line and returned NEEDS_DATA with an explicit discriminating-data list after 180 parameter-defect associations failed false-discovery-rate correction, redirecting the in-

vestigation from parameter tuning toward surface-inspection and equipment-event data; no cause was fabricated. Projected: the integration, sovereignty, and documentation claims below are design properties of the architecture, not yet benchmarked outcomes.

Integration with existing plant infrastructure. IDD consumes what plants already produce: flat exports from historians (OPC-UA/PI), LIMS quality records, and batch logs. The same pipeline runs interactively through a web application and programmatically through a versioned REST API (`/api/diagnosis` with engine selection and typed error contracts), so a deployment is an export job plus one API call rather than an instrumentation project. Because the deterministic stage is byte-reproducible, results do not drift when the surrounding IT landscape changes.

Data-sovereign deployment. The fourteen-engine execution layer (Section 3) accepts on-premise and private-model agents through uniform dialect adapters. For operators who cannot send process data to external LLM services—defence, pharmaceuticals, semiconductors, national infrastructure—the reasoning stage can be pointed at an internally hosted model without touching the audit, evidence, or gate contracts; the benchmark protocol itself executes identically against a scripted mock engine, which is how the full chain is exercised in CI.

Conclusions that route work, not just verdicts. The three-state protocol converts diagnosis into an engineering workflow: DETERMINED conclusions carry mechanism chains that point to corrective actions; COMPETING_SET and NEEDS_DATA conclusions carry named discriminating measurements—in effect, work orders for targeted instrumentation

or tests. The three capped benchmark verdicts illustrate the value directly: each tells the operator exactly which additional channel would have resolved the case.

Documentation for regulated operations. Because every conclusion is bound to graded evidence, physics checks, and a fingerprinted execution trace, the by-product of each run is an RCA report whose traceability serves the documentation expectations of regulated environments (e.g., GxP batch-deviation investigation, in the sense of ALCOA+-style data integrity and GAMP 5 computerised-systems thinking). We state this as a documentation property of the artifact—not as a computerised-system validation claim, which would require a formal qualification exercise.

Deployment governance and knowledge compounding. IDD is positioned as decision support: a DETERMINED verdict does not authorise maintenance action—a human engineer signs off, and the three-state protocol is precisely the escalation interface between machine conclusion and human decision. Model-change management rides the engine layer’s version disclosure (Section 6): a provider silently updating the reasoning model changes the system, so the benchmark configuration disclosure doubles as the deployment pinning record. Finally, the ontology layer is an asset store, not a per-run artifact: variable semantics, physical principles, and confounder annotations published once for a plant or product family are reused by every subsequent diagnosis—in this benchmark, the SKAB and TEP sibling scenarios resolved their ontologies from the published asset store in seconds instead of rebuilding—and every field incident becomes a candidate graded scenario, closing the loop between deployment and evaluation. The marginal cost of a

diagnosis therefore falls as the fleet grows—an economics that sample-level detectors, which must be retrained per asset, do not share.

Scope of applicability. The method requires multivariate time-ordered process data with engineering semantics for the channels (to build the ontology) and applies to continuous, batch, and testbed processes alike—the three benchmark families were chosen to span exactly that range. It does not apply where no time-aligned process data exists, and its root-cause reach is bounded by the sensor set, as the capped verdicts demonstrate honestly.

10. Conclusion

We presented IDD, an agentic diagnosis pipeline that turns LLM-based root-cause analysis into an auditable engineering procedure: competing hypotheses with mechanism classes, evidence levels bounding reported confidence, a four-condition anti-spurious filter, automatic physical verification, and quality gates with bounded repair. We further contributed a machine-checked evaluation contract for diagnosis benchmarks—truth isolation, data fingerprinting, stage-authored artifacts with provenance, gate-graded scoring—and executed the complete pipeline, one recorded execution per scenario, on a 12-scenario corpus across SKAB, TEP, and IndPenSim. IDD resolved 6/9 faults to a single root cause (5/9 under the pre-revision keyword set; Section 5) and ranked the true mechanism among the candidates in two more, with zero false alarms, capped competing-set verdicts with named discriminating measurements where the data could not discriminate, and every result backed by an execution proof (rubric mean 97.9/100, finalisation 12/12 PASS, artifact-consistency verified). On the per-fault TEP compar-

ison it is Top-1 correct on 5/6 without candidate lists (4/6 pre-revision; Section 5)—matching the strongest o1-preview baseline on shared detectable faults, converting the GPT-4o baseline’s IDV1 miss into a resolved diagnosis, and scoring the PCA-excluded pair that baselines exclude—while adding the evidence, compliance, and three-state dimensions those baselines lack. The claims are bounded: results are single recorded executions of a stochastic reasoning layer whose configuration is disclosed (the paper’s own drawn same-era retest makes this concrete: IDV11’s verdict state flipped across sessions while its mechanism held, a 5/9 resolved count under that draw; Section 8.4), the rubric and keywords are author-designed instruments, independent expert anchoring is outstanding, a model-controlled ablation (Section 8.3) shows that a bare same-model call scores ahead of the pipeline on documented faults (9/9 versus 6/9 resolved), and the pipeline’s demonstrated value is therefore the auditable, honestly-uncertain process rather than an accuracy gain. The evidence base is one model family at one deployment on twelve public-dataset scenarios; cross-model and cross-plant generalisation of the process guarantees (in contrast to the accuracy findings, which the ablation already bounds) remains to be demonstrated. The evaluation framework, per-scenario evidence chains, and consistency artifacts are released to support scrutiny and reuse; the pre-registered repeat study, the remaining ablation arms, and novel-disturbance scenarios are the immediate next steps toward a definitive multi-benchmark study.

References

- [1] A. Khan, R. Nahar, H. Chen, G. E. Constante-Flores, C. Li, FaultExplainer: Leveraging large language models for interpretable fault detection and diagnosis, *Computers & Chemical Engineering* 199 (2025) 109152, preprint: arXiv:2412.14492. doi:[10.1016/j.compchemeng.2025.109152](https://doi.org/10.1016/j.compchemeng.2025.109152).
- [2] X. Jiang, H. Xie, J. Wang, Z. Yang, Y. Zhou, L. Yao, Z. Zhu, Agentic AI for safety-aware process monitoring and fault diagnosis: A review, *Processes* 14 (13) (2026) 2112. doi:[10.3390/pr14132112](https://doi.org/10.3390/pr14132112).
- [3] M. Kano, K. Nagao, S. Hasebe, I. Hashimoto, H. Ohno, R. Strauss, B. R. Bakshi, Comparison of multivariate statistical process monitoring methods with applications to the Eastman challenge problem, *Computers & Chemical Engineering* 26 (2) (2002) 161–174. doi:[10.1016/S0098-1354\(01\)00738-4](https://doi.org/10.1016/S0098-1354(01)00738-4).
- [4] S. Tuli, G. Casale, N. R. Jennings, TranAD: Deep transformer networks for anomaly detection in multivariate time series data, *Proceedings of the VLDB Endowment* 15 (6) (2022) 1201–1214. doi:[10.14778/3514061.3514067](https://doi.org/10.14778/3514061.3514067).
- [5] V. Pozdnyakov, A. Kovalenko, I. Makarov, M. Drobyshevskiy, K. Lukyanov, Adversarial attacks and defenses in fault detection and diagnosis: A comprehensive benchmark on the tennessee eastman process, *IEEE Open Journal of the Industrial Electronics Society* 5 (2024) 428–440. doi:[10.1109/OJIES.2024.3401396](https://doi.org/10.1109/OJIES.2024.3401396).

- [6] C. A. Rieth, B. D. Amsel, R. Tran, M. B. Cook, [Additional Tennessee Eastman process simulation data for anomaly detection evaluation](#) (2017). doi:[10.7910/DVN/6C3JR1](#).
URL <https://doi.org/10.7910/DVN/6C3JR1>
- [7] S. J. Qin, Survey on data-driven industrial process monitoring and diagnosis, *Annual Reviews in Control* 36 (2) (2012) 220–234. doi:[10.1016/j.arcontrol.2012.09.004](#).
- [8] E. L. Russell, L. H. Chiang, R. D. Braatz, *Data-driven Methods for Fault Detection and Diagnosis in Chemical Processes*, Springer, London, 2000. doi:[10.1007/978-1-4471-0409-4](#).
- [9] L. H. Chiang, E. L. Russell, R. D. Braatz, *Fault Detection and Diagnosis in Industrial Systems*, Springer, London, 2001. doi:[10.1007/978-1-4471-0347-9](#).
- [10] T. Yuan, S. J. Qin, Root cause diagnosis of plant-wide oscillations using Granger causality, *Journal of Process Control* 24 (2) (2014) 450–459. doi:[10.1016/j.jprocont.2013.11.009](#).
- [11] M. Bauer, J. W. Cox, M. H. Caveness, J. J. Downs, N. F. Thornhill, Finding the direction of disturbance propagation in a chemical process using transfer entropy, *IEEE Transactions on Control Systems Technology* 15 (1) (2007) 12–21. doi:[10.1109/TCST.2006.883234](#).
- [12] J. J. Downs, E. F. Vogel, A plant-wide industrial process control problem, *Computers & Chemical Engineering* 17 (3) (1993) 245–255. doi:[10.1016/0098-1354\(93\)80018-I](#).

- [13] F. Hartung, B. J. Franks, T. Michels, D. Wagner, P. Liznerski, S. Reitherrmann, S. Fellenz, F. Jirasek, M. Rudolph, D. Neider, H. Leitte, C. Song, B. Kloepper, S. Mandt, M. Bortz, J. Burger, H. Hasse, M. Kloft, Deep anomaly detection on Tennessee Eastman process data, *Chemie Ingenieur Technik* 95 (7) (2023) 1077–1082. [doi:10.1002/cite.202200238](https://doi.org/10.1002/cite.202200238).
- [14] J. P. Vieira, V. A. Bauler, R. K. Rosa, D. Silva, Towards a more realistic evaluation of machine learning models for bearing fault diagnosis, *Mechanical Systems and Signal Processing* 258 (2026) 114640. [doi:10.1016/j.ymssp.2026.114640](https://doi.org/10.1016/j.ymssp.2026.114640).
- [15] J. Hendriks, P. Dumond, D. A. Knox, Towards better benchmarking using the CWRU bearing fault dataset, *Mechanical Systems and Signal Processing* 169 (2022) 108732. [doi:10.1016/j.ymssp.2021.108732](https://doi.org/10.1016/j.ymssp.2021.108732).
- [16] M. Iri, K. Aoki, E. O'Shima, H. Matsuyama, An algorithm for diagnosis of system failures in the chemical process, *Computers & Chemical Engineering* 3 (1–4) (1979) 489–493. [doi:10.1016/0098-1354\(79\)80079-4](https://doi.org/10.1016/0098-1354(79)80079-4).
- [17] F. Tao, H. Zhang, A. Liu, A. Y. C. Nee, Digital twin in industry: state-of-the-art, *IEEE Transactions on Industrial Informatics* 15 (4) (2019) 2405–2415. [doi:10.1109/TII.2018.2873186](https://doi.org/10.1109/TII.2018.2873186).
- [18] W. Gong, S. Qiao, C. Cao, S. Tan, J. Ye, H. Liu, S. Chen, X. Wang, Harnessing collective intelligence of multi-agent LLM systems for sensor failure reasoning in smart manufacturing, *Journal of Industrial Information Integration* 49 (2026) 101012. [doi:10.1016/j.jii.2025.101012](https://doi.org/10.1016/j.jii.2025.101012).

- [19] C. Constantinides, D. Patel, S. Lin, C. Guerrero, S. D. Patil, et al., FailureSensorIQ: A multi-choice QA dataset for understanding sensor relationships and failure modes (2025). [arXiv:2506.03278](#), [doi:10.48550/arXiv.2506.03278](#).
- [20] C. Liu, J. Song, D. Tang, L. Wang, H. Zhu, Q. Cai, Probing a novel machine tool fault reasoning and maintenance service recommendation approach through data-knowledge empowered LLMs integrated with AR-assisted maintenance guidance, *Advanced Engineering Informatics* 66 (2025) 103460. [doi:10.1016/j.aei.2025.103460](#).
- [21] Y. Liu, Z. Xu, J. Zhao, C. Song, Z. He, Spatial-temporal adaptive causality graph-based fault root cause location method for time-varying industrial process, *Advanced Engineering Informatics* 68 (2025) 103765. [doi:10.1016/j.aei.2025.103765](#).
- [22] W. Yue, J. Chai, X. Wan, Y. Xie, X. Chen, W. Gui, Root cause analysis for process industry using causal knowledge map under large group environment, *Advanced Engineering Informatics* 57 (2023) 102057. [doi:10.1016/j.aei.2023.102057](#).
- [23] Z. Liu, H. Zheng, H. Liu, W. Jia, J. Tan, Label-free evaluation for performance of fault diagnosis model on unknown distribution dataset, *Advanced Engineering Informatics* 62 (2024) 102912. [doi:10.1016/j.aei.2024.102912](#).
- [24] B. Zhou, X. Li, T. Liu, K. Xu, W. Liu, J. Bao, CausalKGPT: Industrial structure causal knowledge-enhanced large language model for

- cause analysis of quality problems in aerospace product manufacturing, *Advanced Engineering Informatics* 59 (2024) 102333. [doi:10.1016/j.aei.2023.102333](https://doi.org/10.1016/j.aei.2023.102333).
- [25] L. Lin, S. Zhang, S. Fu, Y. Liu, FD-LLM: Large language model for fault diagnosis of complex equipment, *Advanced Engineering Informatics* 65 (2025) 103208. [doi:10.1016/j.aei.2025.103208](https://doi.org/10.1016/j.aei.2025.103208).
- [26] S. Wen, F. Li, W. Zhuang, X. Pan, W. Yu, J. Bao, X. Li, Leveraging large language models for Human-Machine collaborative troubleshooting of complex industrial equipment faults, *Advanced Engineering Informatics* 65 (2025) 103235. [doi:10.1016/j.aei.2025.103235](https://doi.org/10.1016/j.aei.2025.103235).
- [27] Q. Xu, G. Wen, Z. Lei, S. Gu, Y. Su, Z. Zhang, X. Chen, Deep digital twin-powered large vision-language model for multi-scenario industrial fault diagnosis, *Advanced Engineering Informatics* 69 (2026) 103997. [doi:10.1016/j.aei.2025.103997](https://doi.org/10.1016/j.aei.2025.103997).
- [28] A. Wei, O. Fink, AgentRCA: Agentic root cause analysis through evidence-grounded reasoning (2026). [arXiv:2607.22385](https://arxiv.org/abs/2607.22385), [doi:10.48550/arXiv.2607.22385](https://doi.org/10.48550/arXiv.2607.22385).
- [29] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, AutoGen: Enabling next-gen LLM applications via multi-agent conversation (2023). [arXiv:2308.08155](https://arxiv.org/abs/2308.08155), [doi:10.48550/arXiv.2308.08155](https://doi.org/10.48550/arXiv.2308.08155).
- [30] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: Inter-

- national Conference on Learning Representations (ICLR), 2023. doi:
[10.48550/arXiv.2210.03629](https://arxiv.org/abs/10.48550/arXiv.2210.03629).
- [31] R. Parasuraman, T. B. Sheridan, C. D. Wickens, A model for types and levels of human interaction with automation, *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 30 (3) (2000) 286–297. doi:[10.1109/3468.844354](https://doi.org/10.1109/3468.844354).
 - [32] J. D. Lee, K. A. See, Trust in automation: designing for appropriate reliance, *Human Factors* 46 (1) (2004) 50–80. doi:[10.1518/hfes.46.1.50_30392](https://doi.org/10.1518/hfes.46.1.50_30392).
 - [33] R. J. Heuer, *Psychology of Intelligence Analysis*, Center for the Study of Intelligence, Central Intelligence Agency, Washington, DC, 1999.
 - [34] A. B. Hill, The environment and disease: association or causation?, *Proceedings of the Royal Society of Medicine* 58 (1965) 295–300. doi:[10.1177/003591576505800503](https://doi.org/10.1177/003591576505800503).
 - [35] C. K. Chow, On optimum recognition error and reject tradeoff, *IEEE Transactions on Information Theory* 16 (1) (1970) 41–46. doi:[10.1109/TIT.1970.1054406](https://doi.org/10.1109/TIT.1970.1054406).
 - [36] J. Morbach, A. Wiesner, W. Marquardt, OntoCAPE—a (re)usable ontology for computer-aided process engineering, *Computers & Chemical Engineering* 33 (10) (2009) 1546–1556. doi:[10.1016/j.compchemeng.2009.01.019](https://doi.org/10.1016/j.compchemeng.2009.01.019).
 - [37] I. D. Katser, V. O. Kozitsin, Skoltech anomaly benchmark

- (SKAB), <https://www.kaggle.com/dsv/1693952> (2020). doi:10.34740/KAGGLE/DSV/1693952.
- [38] S. Goldrick, A. Ştefan, D. Lovett, G. Montague, B. Lennox, The development of an industrial-scale fed-batch fermentation simulation, *Journal of Biotechnology* 193 (2015) 70–82. doi:10.1016/j.jbiotec.2014.10.029.
- [39] S. Goldrick, C. A. Duran-Villalobos, K. Jankauskas, D. Lovett, S. S. Farid, B. Lennox, Modern day monitoring and control challenges outlined on an industrial-scale benchmark fermentation process, *Computers & Chemical Engineering* 130 (2019) 106471. doi:10.1016/j.compchemeng.2019.05.037.
- [40] A. Iliopoulos, J. Violos, C. Diou, I. Varlamis, Detection of anomalies in multivariate time series using ensemble techniques, in: 2023 IEEE Ninth International Conference on Big Data Computing Service and Applications (BigDataService), 2023, pp. 1–8. doi:10.1109/BigDataService58306.2023.00007.
- [41] C. Reinartz, M. Kulahci, O. Ravn, An extended tennessee eastman simulation dataset for fault detection and decision support systems, *Computers & Chemical Engineering* 149 (2021) 107281. doi:10.1016/j.compchemeng.2021.107281.

Appendix A. Supplementary Tables and Released-Artifact Index

Table A1 transcribes the per-fault TEP comparison discussed in Section 8; Table A2 indexes the artifacts released for every benchmark scenario,

all of which are machine-readable and carry the execution proofs described in Section 5; and Table A3 records the keyword-calibration chronology referenced in Section 5. The machine-generated English benchmark report is `results/benchmark/benchmark_report_en.md`; its own appendix lists the grader-side ground truth and must not be exposed to a diagnosing agent.

Table A1: Per-Fault Outcomes on the TEP Subset (machine-readable companion to Fig. 6). FE = FaultExplainer (root-causes-included prompt, top-3 candidate list, alias hits accepted); “unscored” = excluded from FE scoring by its PCA implementation [9]. IDD: strict mechanism-keyword scoring, no candidate list; CS = capped COMPETING_SET. In FE’s additional no-candidate regime (General-Reasoning prompt, top-3, lenient “related” grading), both models score 8/11. IDD’s Top-1 summary of 5/6 is under the calibrated keyword set; under the pre-revision set IDV1 does not hit (its sole hit is the calibrated set’s added synonym) and the summary is 4/6 (Section 5).

TEP fault	FE GPT-4o	FE ol-preview	PCA detectable	IDD verdict (conf.)
IDV1 A/C feed-ratio step (Stream 4)	wrong	correct	yes	Top-1 correct (0.79)
IDV3 D-feed temperature step	unscored	unscored	no	CS 0.60 (site indistinguishable)
IDV4 reactor cooling-water temp. (step)	unscored	unscored	no	Top-1 correct (0.75)
IDV7 C-header pressure (Stream 4)	correct	correct	yes	Top-1 correct (0.82)
IDV11 cooling-water temp. (random)	correct	correct	yes	Top-1 correct (0.80)
IDV14 cooling-water valve (stiction)	correct	correct	yes	Top-1 correct (0.89)
Scoreable-set summary	7/11	9/11	—	5/6 (CI [43.6, 97.0]); shared detectable subset 4/4
FE no-candidate regime (top-3, lenient)	8/11 both models		—	—

Table A2: Released-artifact index (paths relative to each released run directory `workspace/diagnostic-runs/<ts>_bench_<case>/`).

Artifact	Content
00_input/data.csv	the blind scenario recording as ingested
results/benchmark/briefs/<case_id>.brief.json	blind evidence pack (statistics + process description; leakage-sentinel checked; released beside the runs, not inside each run directory)
01_ontology/ontology.json	process ontology: variables, semantics, physical principles
02_processed/validate_report.json	deterministic statistical validation (correlations, anti-spurious checks)
02_processed/data_analysis_conclusion.json	mandatory handoff to the diagnostician
03_figures/visual_analysis.json	visual-evidence analysis (script-signed in the non-visual deployment)
04_diagnostics/diagnosis.json	verdict, hypotheses, mechanism classes, eliminations
04_diagnostics/evidence.json	evidence inventory with numeric evidence ranks (1–7, the L1–L7 levels)
04_diagnostics/confidence.json	per-hypothesis confidence and ceiling audit
04_diagnostics/reasoning_chain.json	R1–R8 reasoning chain with falsification conditions
05_review/judge_feedback.json	judge sub-agent ten-criterion verdict
05_review/optimizer_preflight.md	pre-report physical audit
05_review/html_review.json	independent HTML review verdict
report.md + diagnostic-report.html	scenario report and reviewed HTML deliverable
optimizer.md	final physical audit containing ENDORSED
.pipeline_events.jsonl	execution event log (finalisation-gate input)
pipeline_finalize_report.json	finalisation gate verdict (overall = PASS required)

Table A3: Keyword-calibration chronology from the run journal and git history (journal stamps in UTC; local time is UTC+8; the calibrated set is pinned at commit 221d061). Keywords are grading-side only and never touch execution inputs.

Step	Record	Time (UTC)
Calibrated set applied	applied once, uniformly, to the working scenario definitions	before the first canonical grading that counted (journal-inferred)
Switch journaled	grading batch re-run under the calibrated set	09-14 20:06–20:09Z
Pre-switch batch	3 fault scenarios + 2 controls graded minutes before the switch completed; 2 outcomes unchanged, TEP IDV1 flips to a hit at 20:09:48Z	09-14 20:06:32Z
Canonical gradings	every later canonical grading uses the calibrated set (first later batch 21:17:16Z)	from 09-14 21:17Z
Canonical executions	the twelve executions (keywords never touch execution inputs); run-directory numerals are UTC	09-14 18:43Z – 09-15 04:54Z
Commit 221d061	formalizes the calibrated set the same day, after these passes	09-15 11:20Z

CRedit authorship contribution statement

ZhangHaoZheng: Conceptualisation, methodology, software, investigation, data curation, writing – original draft, writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The evaluation repository accompanies this submission as supplementary material: scenario definitions with grading keywords, expected verdict sets and per-fault literature baselines (with edit history), per-scenario blind inputs and SHA-256 fingerprints, full pipeline run directories with stage-authored artifacts and execution-event logs, grading records with provenance labels and per-scenario repair counts, the rubric and grading implementations, aggregation and consistency-verification scripts, and the Markdown/HTML scoring reports. An archived snapshot with a persistent identifier (Zenodo DOI) will be minted on acceptance. Third-party datasets are publicly available from their cited sources.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used AI-assisted coding and writing tools in order to develop the evaluation harness, execute the benchmark pipeline, and support manuscript drafting and editing. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. The agentic system that is the *subject* of this paper (IDD) is unrelated to the writing-assistance tools used during preparation; all benchmark executions reported here were produced by the IDD pipeline itself under the released evaluation harness.